

EAFC UCCLE - Établissement d'Enseignement pour Adultes et de Formation Continue  
Travail de fin d'études - Bachelier en informatique de gestion

# ENGIE Queries

## Travail de fin d'études en vue de l'obtention du titre de bachelier en informatique de gestion

*Développement d'une application Python sécurisée permettant la définition de requêtes sur base de  
filtres (critère de sélection) dans le cadre du reporting et audit des activités de trading.*

**Réalisé par :** Ters Marwa

**Chargée de cours :** Sonneville Véronique

**Année académique :** 2025-2026

## REMERCIEMENTS

---

Réaliser ce travail de fin d'études n'aurait tout simplement pas été possible sans l'aide et le soutien de nombreuses personnes, et je tiens à leur exprimer ici toute ma gratitude.

Je remercie tout d'abord Madame Sonnevile Véronique, chargée de cours à l'EAFC Uccle, pour son accompagnement bienveillant tout au long de ce projet. Ses conseils méthodologiques et sa disponibilité ont été déterminants pour structurer ce travail et m'aider à aller au bout.

Je souhaite également remercier l'ensemble des professeurs qui m'ont accompagnée durant ces années de bachelier. C'est grâce à leurs enseignements et à leur passion pour l'informatique que j'ai pu acquérir les bases nécessaires pour mener à bien un projet d'une telle envergure.

Un grand merci à l'équipe Restitution du département Deal and Risk Management d'ENGIE, et plus particulièrement à mon encadrant Benjamin Beke, pour m'avoir accueillie chaleureusement et offert un environnement de travail stimulant et bienveillant. Cette expérience a été bien plus qu'un simple stage, elle m'a permis de grandir professionnellement et de découvrir les réalités du monde de l'entreprise.

Enfin, je remercie du fond du cœur ma famille et mes proches pour leur soutien indéfectible, leur patience et leurs encouragements tout au long de mes études. Leur présence a été une force précieuse.

À toutes ces personnes, merci.

# SOMMAIRE

|                                                             |           |
|-------------------------------------------------------------|-----------|
| <b>1. INTRODUCTION</b>                                      | <b>5</b>  |
| ■ 1.1 Contexte du projet                                    | 5         |
| ■ 1.2 Présentation du stage                                 | 6         |
| ■ 1.3 Problématique                                         | 6         |
| <b>2. OBJECTIFS DU PROJET</b>                               | <b>7</b>  |
| ■ 2.1 Objectifs fonctionnels                                | 7         |
| ■ 2.2 Objectifs techniques                                  | 7         |
| ■ 2.3 Apport personnel                                      | 7         |
| ■ 2.4 Méthodologie                                          | 8         |
| ■ 2.5 Conventions de messages de commit                     | 9         |
| <b>3. DOSSIER D'ANALYSE</b>                                 | <b>10</b> |
| ■ 3.1 Cahier des charges                                    | 10        |
| 3.1.1 Règles de gestion                                     | 10        |
| 3.1.2 Catégories d'utilisateurs                             | 11        |
| 3.1.3 Diagramme de cas d'utilisation                        | 13        |
| 3.1.4 Fonctionnalités implémentées                          | 13        |
| ■ 3.2 Persistance des données                               | 15        |
| 3.2.1 Diagramme de classes                                  | 15        |
| 3.2.2 Dictionnaire des données                              | 18        |
| 3.2.3 Collections MongoDB                                   | 20        |
| ■ 3.3 Autres diagrammes                                     | 24        |
| 3.3.1 Diagramme de séquence d'Authentification JWT          | 24        |
| 3.3.2 Diagramme de séquence de Reporting automatique en PDF | 26        |
| 3.3.3 Diagramme d'activité                                  | 27        |
| 3.3.4 BPMN                                                  | 28        |
| <b>4. DOSSIER TECHNIQUE</b>                                 | <b>29</b> |
| ■ 4.1 Technologies utilisées                                | 29        |
| 4.1.1 Vue d'ensemble de l'architecture                      | 29        |
| 4.1.2 Frontend Angular                                      | 29        |
| 4.1.3 Backend FastAPI                                       | 31        |
| 4.1.4 Base de données MongoDB                               | 33        |
| 4.1.5 Sécurité JWT/RBAC                                     | 33        |

|                                                              |           |
|--------------------------------------------------------------|-----------|
| 4.1.6 Outils de développement.....                           | 34        |
| ■ 4.2 Automatisation : Reporting et alertes .....            | 35        |
| 4.2.1 Rapport mensuel automatique .....                      | 36        |
| 4.2.2 Rapport instantané lors d'une désactivation.....       | 36        |
| 4.2.3 Configuration SMTP .....                               | 36        |
| ■ 4.3 Mécanismes techniques clés .....                       | 37        |
| 4.3.1 Exécution dynamique des queries .....                  | 37        |
| 4.3.2 Filtres MongoDB et récupération des données .....      | 37        |
| 4.3.3 Génération automatique des audits .....                | 38        |
| 4.3.4 Historisation des anciennes et nouvelles valeurs ..... | 39        |
| 4.3.5 Sauvegarde des filtres en JSON.....                    | 39        |
| ■ 4.4 Gestion des rôles et permissions.....                  | 40        |
| 4.4.1 Rôle Administrateur .....                              | 40        |
| 4.4.2 Rôle Owner .....                                       | 40        |
| 4.4.3 Rôle Member .....                                      | 40        |
| 4.4.4 Gestion des permissions dans les groupes .....         | 40        |
| 4.4.5 Sécurité des échanges frontend/backend .....           | 41        |
| 4.4.6 Protection des routes Angular .....                    | 41        |
| ■ 4.5 Interfaces et fonctionnalités.....                     | 42        |
| 4.5.1 Interface d'authentification .....                     | 42        |
| 4.5.2 Interface de gestion des utilisateurs.....             | 43        |
| 4.5.3 Interface de gestion des groupes.....                  | 44        |
| 4.5.4 Interface de gestion des memberships.....              | 45        |
| 4.5.5 Interface de gestion des queries.....                  | 46        |
| 4.5.6 Interface d'exécution des queries et résultats .....   | 47        |
| 4.5.7 Interface d'audit et de traçabilité .....              | 47        |
| 4.5.8 Documentation Swagger.....                             | 48        |
| ■ 4.6 Tests fonctionnels.....                                | 48        |
| ■ 4.7 Difficultés rencontrées .....                          | 50        |
| <b>5. CONCLUSION.....</b>                                    | <b>51</b> |
| <b>6. BIBLIOGRAPHIE .....</b>                                | <b>52</b> |
| <b>TABLE DES FIGURES.....</b>                                | <b>53</b> |
| <b>LISTE DES TABLEAUX .....</b>                              | <b>55</b> |

# 1. INTRODUCTION

---

Dans le cadre de l'obtention du titre de bachelier en informatique de gestion, j'ai réalisé mon travail de fin d'études au sein d'ENGIE Belgium, l'un des leaders mondiaux du secteur énergétique. Ce stage s'est déroulé du 15 septembre au 30 novembre 2025, au sein de l'équipe Restitution du département Deal and Risk Management d'ENGIE Global Markets à Bruxelles.

Le secteur du trading énergétique est un environnement particulièrement exigeant, où les décisions doivent être prises rapidement sur des marchés volatils. Les traders négocient quotidiennement des matières premières telles que le gaz naturel, l'électricité, le pétrole et les certificats CO<sub>2</sub>, et ont besoin d'outils fiables pour analyser leurs données et prendre les meilleures décisions possible.

Lorsque j'ai intégré l'équipe, une application Python<sup>1</sup> existait déjà. Elle permettait l'authentification par token et la création de requêtes filtrées sur les données de deals. Cependant, plusieurs lacunes importantes avaient été identifiées : absence de gestion des groupes d'utilisateurs, manque de traçabilité des actions, absence de reporting automatisé et sécurité insuffisante.

Mon travail a donc consisté à reconcevoir cette application en partant de zéro, tout en conservant le principe des filtres comme fonctionnalité centrale. J'y ai intégré une gestion complète des groupes d'utilisateurs, un système d'audit et de traçabilité, un reporting automatisé avec génération de rapports PDF et envoi d'alertes par email, ainsi qu'une interface web complète développée en Angular.

Ce rapport documente l'ensemble de la démarche suivie, depuis l'analyse des besoins jusqu'à la réalisation technique. Il est destiné au jury académique qui évaluera la qualité du travail, et s'organise autour des grandes parties suivantes : le dossier d'analyse, le dossier technique et la conclusion.

## 1.1 Contexte du projet

Dans le domaine du trading énergétique, les traders manipulent quotidiennement des informations sensibles relatives aux deals : produits échangés, volumes, prix, contreparties commerciales, dates de livraison et statuts des opérations. Ces données doivent être analysées rapidement pour prendre les meilleures décisions possible.

Avant ce projet, l'application existante permettait déjà de filtrer ces données et de générer des résultats de requêtes. Cependant, elle manquait de plusieurs fonctionnalités essentielles : les requêtes n'étaient pas organisées de façon collaborative, il n'existait aucun historique des actions réalisées, et la sécurité des accès n'était pas suffisamment structurée.

C'est pour répondre à ces besoins concrets que ce projet a été développé. L'application conçue repose sur une architecture moderne en trois couches : un frontend Angular pour l'interface utilisateur, un

---

<sup>1</sup>Python 3.11 : Documentation officielle. Disponible sur : <https://docs.python.org/3.11>

backend FastAPI<sup>2</sup> en Python pour la logique métier, et une base de données MongoDB<sup>3</sup> pour la persistance des données.

## 1.2 Présentation du stage

Mon stage s'est déroulé au sein d'ENGIE Global Markets, dans l'équipe Restitution du département Deal and Risk Management. Cette équipe a pour mission principale de fournir aux traders et aux analystes des outils fiables pour accéder et exploiter les données liées aux activités de trading énergétique. Elle collabore au quotidien avec les différentes équipes techniques et métier de l'entreprise.

Le projet a été mené selon la méthodologie Agile Scrum, rythmée par des réunions quotidiennes, des sprints de développement, un backlog de tâches priorisées ainsi que des points réguliers avec le Product Owner et mon encadrant Benjamin Beke.

Pour mener à bien ce projet, j'ai utilisé plusieurs outils professionnels :

- Visual Studio Code comme environnement de développement,
- Git pour la gestion des versions,
- Postman pour tester les endpoints de l'API,
- Swagger pour la documentation des APIs REST,
- MongoDB Compass pour visualiser les données,
- Microsoft Teams pour la communication avec l'équipe.

Cette expérience m'a permis de découvrir concrètement ce que signifie travailler en entreprise, notamment en ce qui concerne les contraintes de sécurité, les exigences de qualité du code et l'importance de la communication au sein d'une équipe technique.

## 1.3 Problématique

Dans le cadre des activités de trading énergétique, les utilisateurs doivent quotidiennement effectuer des recherches complexes sur un grand volume de données afin d'analyser les opérations réalisées sur les marchés. Ces recherches nécessitent des filtres variés portant sur le produit énergétique, les dates, les volumes, les prix, les contreparties ou encore les portefeuilles de trading.

Avant le développement de cette application, plusieurs limitations importantes avaient été identifiées dans le système existant :

- absence de centralisation des queries : les requêtes n'étaient pas organisées dans une structure collaborative, ce qui compliquait leur partage entre utilisateurs,
- gestion limitée des droits d'accès : le système ne permettait pas de contrôler précisément les permissions selon les rôles,
- manque de traçabilité : les actions réalisées dans l'application n'étaient pas historisées, rendant impossible le suivi des modifications,

---

<sup>2</sup>FastAPI : Documentation officielle. Disponible sur : <https://fastapi.tiangolo.com>

<sup>3</sup>MongoDB : Documentation officielle. Disponible sur : <https://www.mongodb.com/docs>

- difficulté de collaboration : aucun système de groupes ne permettait aux utilisateurs de travailler ensemble autour des queries,
- sécurité insuffisante : certaines opérations critiques n'étaient pas suffisamment protégées.

Il était donc nécessaire de développer une solution complète permettant d'organiser les queries, de faciliter la collaboration, de contrôler précisément les accès et de garantir une traçabilité totale des actions réalisées dans l'application.

## 2. OBJECTIFS DU PROJET

---

Ce projet avait pour ambition de répondre concrètement aux besoins identifiés lors de l'analyse de l'application existante. Les objectifs se divisent en deux catégories :

### 2.1 Objectifs fonctionnels

- Mettre en place une authentification sécurisée des utilisateurs via token JWT.
- Gérer différents rôles et niveaux de permissions selon le profil de chaque utilisateur.
- Permettre la création et la gestion de groupes collaboratifs.
- Ajouter ou supprimer des membres au sein d'un groupe.
- Créer, modifier et exécuter des queries d'analyse sur les deals énergétiques.
- Rechercher des deals à l'aide de filtres dynamiques et flexibles.
- Mettre en place un système complet d'audit et de traçabilité des actions.
- Générer automatiquement un rapport PDF d'audit le dernier jour de chaque mois et l'envoyer par email au propriétaire du groupe. Générer également un rapport instantané lors de la désactivation d'un groupe.

### 2.2 Objectifs techniques

- Développer une architecture moderne et modulaire en trois couches.
- Séparer clairement le frontend Angular et le backend FastAPI.
- Exposer des APIs REST documentées et testables via Swagger.
- Sécuriser les données grâce à JWT et au contrôle d'accès basé sur les rôles.
- Concevoir une application évolutive et facile à maintenir.
- Exploiter MongoDB pour une gestion flexible des données.

### 2.3 Apport personnel

Lorsque j'ai débuté ce projet, l'application existait déjà. Elle proposait des fonctionnalités de base comme l'authentification des utilisateurs, la création simple de queries et une gestion basique des deals.

Mon travail a consisté à transformer cette base en une application complète et professionnelle. Concrètement, j'ai développé les fonctionnalités suivantes :

- La gestion complète des groupes d'utilisateurs et des memberships.

- Un système de permissions basé sur les rôles RBAC (administrateur, owner, member).
- Un système d'audit et de traçabilité enregistrant toutes les actions importantes.
- La génération automatique de rapports PDF mensuels et instantanés avec envoi par email au propriétaire du groupe via SMTP.
- L'amélioration des filtres de recherche des deals énergétiques.
- Le développement intégral du frontend Angular.
- La sécurisation avancée des endpoints backend.
- La structuration globale de l'architecture de l'application.
- La création du rôle Administrateur avec des droits de supervision globale, absent de l'application d'origine.

Par rapport à l'application existante au sein d'ENGIE, j'ai également introduit la notion de rôle Administrateur qui n'existait pas dans le système d'origine. Cette distinction entre un administrateur global et un utilisateur standard permet de structurer les droits d'accès de façon plus claire et plus sécurisée, en réservant les opérations sensibles comme la gestion des utilisateurs ou la supervision de tous les groupes à un profil dédié.

Au final, ce projet m'a permis de reconcevoir une application web sécurisée en repartant d'un modèle réel chez ENGIE, pour en faire une solution complète, collaborative et adaptée aux exigences d'un environnement professionnel.

## 2.4 Méthodologie

Le développement de ce projet de fin d'études a suivi la même méthodologie que celle adoptée durant le stage au sein d'ENGIE, à savoir l'approche Agile Scrum. Le travail a été organisé en sprints successifs, chacun comprenant une phase d'analyse, de développement, de tests et de corrections.

Les mêmes outils professionnels utilisés durant le stage ont été conservés : Git<sup>4</sup> et GitHub pour la gestion des versions et le suivi des modifications, Postman<sup>5</sup> pour tester les endpoints de l'API, et Swagger<sup>6</sup> pour valider la documentation des routes REST.

Cette continuité méthodologique m'a permis de maintenir une organisation rigoureuse tout au long du développement et de travailler de façon structurée et professionnelle.

---

<sup>4</sup>Git : Documentation officielle. Disponible sur : <https://git-scm.com/doc>

<sup>5</sup>Postman : Outil de test des APIs. Disponible sur : <https://www.postman.com>

<sup>6</sup>Swagger UI : Documentation interactive des APIs REST. Disponible sur : <https://swagger.io/tools/swagger-ui>



| A6 |    |                                                                                                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |                |                 |             |        |                                     |                        |
|----|----|-------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|-----------------|-------------|--------|-------------------------------------|------------------------|
|    | A  | B                                                                                                                 | C                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | D              | E               | F           | G      | H                                   | I                      |
| 1  | ID | User Story                                                                                                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | Estimation J/h | Réalisation J/h | Ecart (+/-) | Sprint | Statut (To Do / In Progress / Done) | Critères d'acceptation |
| 1  |    |                                                                                                                   | Back:<br>1 API d'authentification<br>2 BDD: Table user avec rôle loginmdp<br>3 CRUD user:<br>API user: endpoint création<br>API user: endpoint update<br>API user: endpoint DELETE<br>API user: endpoint GET (affichage)<br><br>Front:<br>1 Ecran d'authentification<br>2 Ecran d'accueil Admin avec les fonctionnalités suivantes:<br>User<br>Groupe<br>Membre<br>Audit<br>Déconnexion<br>3 Ecran avec la liste des users :<br>Liste des users<br>Barre de Recherche<br>Création nouveau user<br>Modif<br>Suppression<br>Consultation (écran à développer: information du ser) | 2              |                 |             | 1      |                                     |                        |
| 2  |    | En tant qu'Admin, je veux me connecter avec un login et mot de passe afin de créer/modifier/supprimer les users   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |                |                 |             |        | To Do                               |                        |
| 3  |    |                                                                                                                   | Back:<br>1 API groupe: endpoint création<br>2 API groupe: endpoint update<br>3 API groupe: endpoint DELETE<br>4 API groupe: endpoint GET (affichage)<br>5 API audit: endpoint UPDATE (action sur le groupe par l'Admin)<br><br>Front:<br>1 Ecran de gestion des groupes<br>Liste des groupes<br>Barre de Recherche<br>Création nouveau groupe<br>Modif<br>Suppression<br>Consultation (écran membres des groupes) Etape suivante)                                                                                                                                               | 2              |                 |             | 1      |                                     |                        |
| 3  |    | En tant qu'Admin, je veux me connecter avec un login et mot de passe afin de créer/modifier/supprimer les groupes |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |                |                 |             |        | To Do                               |                        |
|    |    |                                                                                                                   | Back:<br>1 API MEMBRE: endpoint création<br>2 API MEMBRE: endpoint DELETE<br>3 API MEMBRE: endpoint GET (affichage)                                                                                                                                                                                                                                                                                                                                                                                                                                                             |                |                 |             |        |                                     |                        |

Figure 1 : Capture du planning des sprints et du backlog utilisés dans le cadre de la méthodologie Agile Scrum

## 2.5 Conventions de messages de commit

Dans le cadre de cette gestion de versions, les messages de commit suivent la convention Conventional Commits. Chaque message respecte la structure type(portée) : description, où le type qualifie la nature de la modification, la portée (optionnelle) précise la zone du code concernée, et la description résume le changement de façon concise. Cette normalisation rend l'historique clair et exploitable, facilite la relecture et la recherche, et permet de comprendre rapidement l'évolution du projet. Les principaux types utilisés sont les suivants :

- feat : ajout d'une nouvelle fonctionnalité (ex. : création de la gestion des groupes).
- fix : correction d'un bug (ex. : conversion des ObjectId MongoDB en JSON).
- docs : modification de la documentation uniquement (README, commentaires, Swagger).
- style : changements de forme sans impact fonctionnel (indentation, formatage, espaces).
- refactor : réécriture du code sans ajout de fonctionnalité ni correction de bug.
- perf : amélioration des performances (optimisation d'une requête, d'un filtre).
- test : ajout ou modification de tests.
- chore : tâches de maintenance et de configuration (dépendances, scripts, .env, CI).
- build : modifications du système de build ou des dépendances externes.
- ci : modifications de la configuration et des scripts d'intégration continue.

Quelques exemples concrets tirés de l'historique GitHub du projet :

- feat(user-form): add confirm password field validation : ajout d'une nouvelle fonctionnalité.
- fix(login): add ENGIE logo above login card : correction d'un problème d'affichage.
- refactor(navbar): apply CSS variables : réorganisation du code sans changement de comportement.
- revert(design): restore original CSS styles : retour à un état antérieur du code.

Les fonctionnalités étaient développées sur des branches dédiées (par exemple feature/audit-reporting) puis intégrées à la branche principale via des pull requests (par exemple « Merge pull request #1 »).

## 3. DOSSIER D'ANALYSE

---

### 3.1 Cahier des charges

#### 3.1.1 Règles de gestion

L'application repose sur un ensemble de règles métier qui définissent comment les données sont organisées, qui peut faire quoi, et comment les actions sont tracées.

Gestion des utilisateurs

- Chaque utilisateur possède un compte unique identifié par son adresse e-mail.
- Chaque utilisateur dispose d'un rôle définissant son niveau de permission dans l'application.
- Les administrateurs ont accès à l'ensemble des fonctionnalités de la plateforme.

Gestion des groupes

- Un groupe peut être créé par n'importe quel utilisateur authentifié.
- Le créateur d'un groupe en devient automatiquement le propriétaire (owner).
- Le propriétaire peut ajouter ou supprimer des membres et gérer les queries du groupe.
- Un utilisateur peut appartenir à plusieurs groupes simultanément.
- La suppression d'un groupe est logique : le groupe est désactivé mais les données sont conservées.

Gestion des memberships

- L'appartenance d'un utilisateur à un groupe est matérialisée par un membership.
- Un membership peut être ajouté ou supprimé selon les permissions de l'utilisateur connecté.
- Deux rôles existent au niveau d'un groupe : OWNER et MEMBER.

Gestion des queries

- Une query appartient obligatoirement à un groupe.
- Les membres du groupe peuvent consulter et exécuter les queries associées.
- La modification ou la suppression d'une query est limitée aux membres autorisés.
- Les queries ne stockent pas de données mais uniquement des filtres appliqués dynamiquement aux deals.

## Gestion des deals

- Les deals représentent les opérations de trading énergétique stockées en base de données.
- Ils peuvent être filtrés dynamiquement selon le produit, les dates, les prix, les volumes, les contreparties, les portefeuilles et les statuts métier.

## Gestion des permissions

- L'application utilise un système RBAC (Role-Based Access Control) avec trois niveaux : Administrateur, Owner et Member.
- Les permissions varient selon le rôle global de l'utilisateur et son rôle au sein du groupe concerné.

## Gestion des audits

- Toutes les actions importantes sont automatiquement historisées dans une collection dédiée.
- Chaque entrée d'audit contient l'utilisateur responsable, le type d'action, l'horodatage, l'entité concernée ainsi que les anciennes et nouvelles valeurs.
- Les logs d'audit sont en lecture seule et ne peuvent pas être modifiés.
- Un rapport PDF est généré automatiquement le dernier jour de chaque mois et envoyé par email au propriétaire du groupe. De plus, lors de la désactivation d'un groupe, un rapport instantané est immédiatement généré et envoyé par email au propriétaire afin de l'informer de l'action réalisée.

## Sécurité

- Toute action dans l'application nécessite un token JWT valide.
- Un token expiré ou invalide est automatiquement refusé par le backend.
- Les mots de passe sont stockés sous forme hashée via bcrypt.

### 3.1.2 Catégories d'utilisateurs

L'application repose sur un système de gestion des accès basé sur les rôles, appelé RBAC<sup>7</sup> (Role-Based Access Control). Trois catégories d'utilisateurs ont été définies, chacune disposant de permissions spécifiques.

#### Administrateur (ADMIN)

L'administrateur possède les droits les plus élevés dans l'application. Il dispose d'un accès global à l'ensemble des fonctionnalités et peut intervenir sur toutes les ressources du système. Ses principales responsabilités sont les suivantes :

- Gérer les comptes utilisateurs (création, modification, suppression).
- Consulter et gérer l'ensemble des groupes de la plateforme.
- Superviser les queries et accéder à tous les audits.
- Assurer le contrôle global de la sécurité et de la maintenance de l'application.

#### Propriétaire du groupe (OWNER)

---

<sup>7</sup>RBAC : Role-Based Access Control. Disponible sur : <https://auth0.com/docs/manage-users/access-control/rbac>

Lorsqu'un utilisateur crée un groupe, il en devient automatiquement le propriétaire. Ce rôle lui confère des droits de gestion sur son espace collaboratif. Il peut notamment :

- Modifier les informations du groupe.
- Ajouter ou supprimer des membres.
- Gérer les queries associées au groupe.
- Consulter l'historique des audits liés au groupe.
- Recevoir les rapports PDF d'audit mensuels et les alertes en cas de désactivation du groupe.

#### Membre (MEMBER)

Le membre est un utilisateur appartenant à un ou plusieurs groupes. Il dispose de droits plus limités que le propriétaire, mais peut néanmoins participer activement aux activités collaboratives du groupe :

- Consulter les queries du groupe.
- Créer et modifier des queries selon les permissions définies.
- Exécuter des recherches sur les deals énergétiques.
- Consulter les audits du groupe dont il est membre.

Tableau 1 : Tableau des permissions des utilisateurs

| Fonctionnalité         | Admin | Owner  | Member |
|------------------------|-------|--------|--------|
| S'authentifier         | ✓     | ✓      | ✓      |
| Consulter les groupes  | ✓     | ✓      | ✓      |
| Créer un groupe        | ✓     | ✓      | ✓      |
| Modifier un groupe     | ✓     | ✓      | ✗      |
| Désactiver un groupe   | ✓     | ✓      | ✗      |
| Ajouter des membres    | ✓     | ✓      | ✗      |
| Supprimer des membres  | ✓     | ✓      | ✗      |
| Créer des queries      | ✗     | Limité | Limité |
| Modifier des queries   | ✗     | Limité | Limité |
| Supprimer des queries  | ✗     | Limité | Limité |
| Exécuter des queries   | ✗     | ✓      | ✓      |
| Consulter les audits   | ✓     | ✓      | Limité |
| Gérer les utilisateurs | ✓     | ✗      | ✗      |

### 3.1.3 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation ci-dessous illustre l'ensemble des interactions entre les différents acteurs et les fonctionnalités de l'application.

Ce dernier met notamment en évidence :

- les opérations accessibles aux administrateurs,
- les fonctionnalités disponibles pour les owners,
- ainsi que les actions autorisées pour les membres.

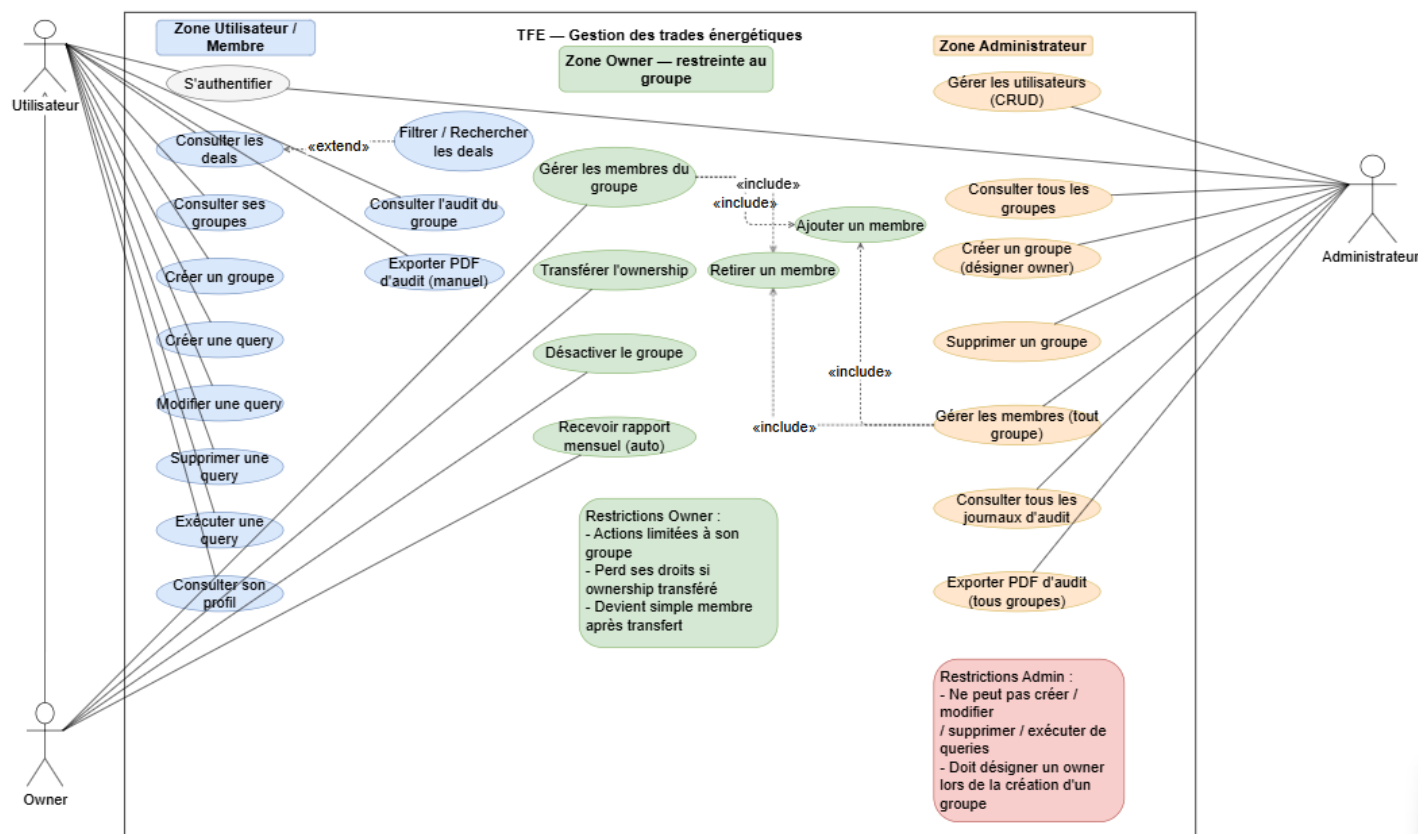


Figure 2 : Diagramme de cas d'utilisation illustrant les fonctionnalités accessibles aux utilisateurs, owners et administrateurs de l'application

### 3.1.4 Fonctionnalités implémentées

L'application développée intègre plusieurs fonctionnalités permettant de répondre aux besoins métier et techniques identifiés lors de l'analyse du projet.

#### Authentification sécurisée

L'application intègre un système d'authentification basé sur les tokens JWT<sup>8</sup> (JSON Web Token). Chaque utilisateur doit se connecter à l'aide de son adresse e-mail et de son mot de passe afin d'accéder aux

<sup>8</sup>JWT : Introduction aux JSON Web Tokens. Disponible sur : <https://jwt.io/introduction>

fonctionnalités sécurisées de la plateforme. Ce mécanisme permet de sécuriser les échanges entre le frontend et le backend, d'identifier l'utilisateur connecté et de contrôler ses permissions d'accès.

### **Gestion des utilisateurs**

Les administrateurs peuvent créer, modifier et supprimer des comptes utilisateurs, consulter les profils et gérer les rôles. Chaque utilisateur possède un rôle, une adresse e-mail, des informations personnelles et un historique d'activité.

### **Gestion des groupes**

L'application permet la création de groupes collaboratifs afin d'organiser les utilisateurs autour des queries et des analyses de deals. Chaque groupe possède un propriétaire, plusieurs membres, des queries associées et un historique d'audit. Les groupes facilitent la collaboration et le partage des analyses entre utilisateurs.

### **Gestion des memberships**

Le système de memberships permet de gérer l'appartenance des utilisateurs aux groupes. Les propriétaires et les administrateurs peuvent ajouter ou supprimer des membres et visualiser les utilisateurs appartenant à un groupe. Cette fonctionnalité garantit une organisation structurée des accès et des permissions.

### **Gestion des queries**

Les utilisateurs peuvent créer et exécuter des queries permettant de rechercher des deals énergétiques selon différents critères de filtrage :

- Le produit énergétique (GAS, ELECTRICITY, OIL, CO2).
- Les dates de début et de fin.
- Les prix minimum et maximum.
- Les volumes.
- Les contreparties commerciales.
- Les portefeuilles de trading.

### **Système d'audit et de traçabilité**

Toutes les actions importantes réalisées dans l'application sont automatiquement enregistrées dans une collection dédiée. Chaque entrée d'audit contient l'utilisateur responsable, le type d'action, l'horodatage, l'entité concernée ainsi que les anciennes et nouvelles valeurs. Les logs d'audit sont consultables via une interface dédiée et ne peuvent pas être modifiés.

### **Reporting automatique**

Un rapport PDF d'audit est généré automatiquement le dernier jour de chaque mois et envoyé par email au propriétaire du groupe. De plus, lors de la désactivation d'un groupe, un rapport instantané est immédiatement généré et envoyé par email au propriétaire afin de l'informer de l'action réalisée.

audit\_final\_med\_group\_10-05-2026.pdf

Journal d'audit  
Groupe : med\_group  
Période : Suppression du groupe — 10/05/2026 · Généré le 10/05/2026 21:14

| Date             | Action       | Cible                 | Utilisateur | Anciennes valeurs | Nouvelles valeurs                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|------------------|--------------|-----------------------|-------------|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 10/05/2026 19:14 | DEACTIVATE   | GROUP - med_group     | mamatin     | status: True      | status: False                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| 10/05/2026 19:13 | CREATE_QUERY | QUERY - test_med      | mamatin     | —                 | query_name: test_med filters: { "product": "CO2", "deal_type": "BUY", "start_date": "2026-04-01", "end_date": "2026-04-30" } group_name: med_group selected_fields: { "DealId": true, "TradeDate": true, "Portfolio": true, "Desk": false, "Entity": true, "Direction": false, "Quantity": true, "QuantityUnit": false, "DeliveryPoint": true, "CounterpartyName": true, "TransportCorridor": false, "DeliveryType": false, "DealType": true, "TraderCode": false, "Price": true, "Cash": false, "OpenQuantity": true, "BookingStatus": false, "MarginCost": false, "TotalMarginCost": false, "BusinessUnit": false } |
| 10/05/2026 19:12 | ADD_MEMBER   | MEMBERSHIP - kmarchal | mamatin     | —                 | group_name: med_group member_username: kmarchal member_email: kevin@example.com role: MEMBER                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| 10/05/2026 19:12 | ADD_MEMBER   | MEMBERSHIP - malucas  | mamatin     | —                 | group_name: med_group member_username: malucas member_email: lucas@example.com role: MEMBER                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| 10/05/2026 19:11 | CREATE       | GROUP - med_group     | mamatin     | —                 | name: med_group description: aaa owner_id: 6a00d8347e68229de133c80a status: True                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |

Page 1 - ENGIE Queries - med\_group

Figure 3 : Rapport PDF généré automatiquement par l'application

## 3.2 Persistance des données

La persistance des données constitue un élément central de l'application. Elle permet de stocker, organiser et exploiter les différentes informations manipulées dans le cadre des analyses de deals énergétiques. L'application repose sur une base de données NoSQL MongoDB, choisie pour sa flexibilité, son évolutivité et ses performances adaptées aux recherches dynamiques utilisées dans les queries. Six collections principales ont été définies : users, groups, memberships, queries, deals et audits.

### 3.2.1 Diagramme de classes

Le diagramme de classes ci-dessous illustre la structure globale des entités de l'application ainsi que les relations existantes entre elles. Il permet de visualiser les différentes entités métier, leurs attributs principaux, les relations entre les objets et les cardinalités associées. Le modèle de données a été conçu dans une logique de modularité et de séparation claire des responsabilités afin de faciliter la maintenance et l'évolution future de l'application.

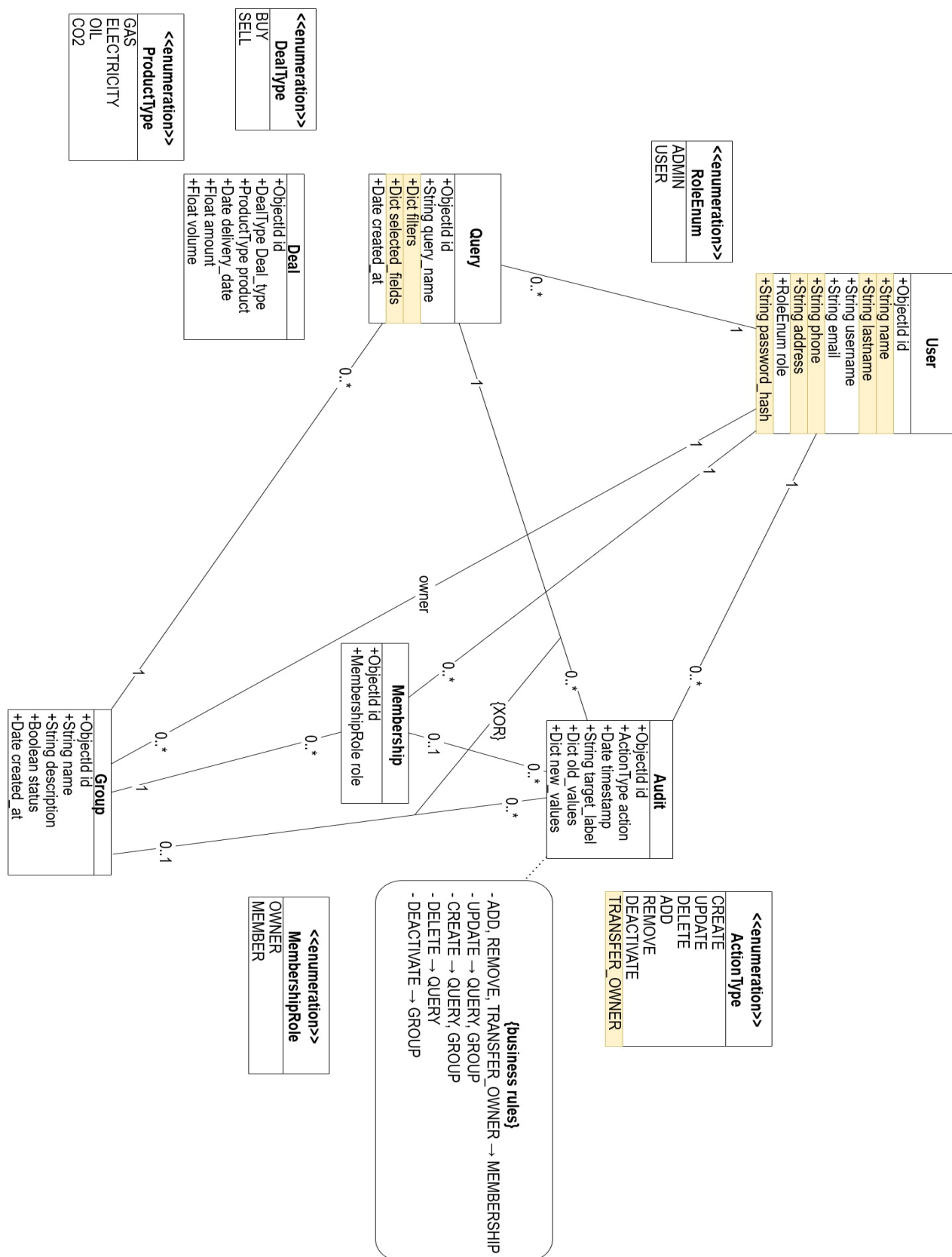


Figure 4 : Diagramme UML de la structure de l'application représentant les principales entités du système



**Relation User → Query :**

Un utilisateur peut créer plusieurs queries. Chaque query possède exactement un créateur, identifié par le champ `created_by`, renseigné automatiquement par le backend à partir du token JWT au moment de la création.

**Relation User → Membership :**

Un utilisateur peut appartenir à plusieurs groupes simultanément. Chaque membership est obligatoirement lié à un seul utilisateur.

**Relation User → Audit :**

Chaque action réalisée par un utilisateur dans l'application génère automatiquement une entrée d'audit. Un audit est toujours associé à un seul utilisateur responsable de l'action.

**Relation Group → Membership :**

Un groupe peut contenir plusieurs membres. Chaque membership appartient obligatoirement à un seul groupe. C'est cette entité intermédiaire qui matérialise la relation entre les utilisateurs et les groupes.

**Relation Group → Query :**

Un groupe peut être associé à plusieurs queries créées en son nom. Chaque query appartient obligatoirement à un seul groupe, ce qui permet d'organiser les analyses autour d'espaces collaboratifs définis.

**Relation Group → Audit :**

Toutes les opérations réalisées au sein d'un groupe sont automatiquement tracées. Chaque audit est rattaché à un seul groupe, permettant de consulter l'historique complet des activités par groupe.

**Relation Group → User (owner) :**

Un utilisateur peut être propriétaire d'un ou plusieurs groupes. Chaque groupe possède exactement un propriétaire, désigné par le champ `owner_id`. Lorsqu'un utilisateur crée un groupe, il en devient automatiquement le propriétaire.

**Contrainte {XOR} sur les audits :**

Les relations entre Audit et les entités Query, Membership et Group sont soumises à une contrainte XOR. Cela signifie que chaque entrée d'audit cible une et une seule entité à la fois, déterminée par le champ **target\_type** :

- GROUP → pour les actions liées aux groupes : CREATE, UPDATE, DEACTIVATE.
- QUERY → pour les actions liées aux queries : CREATE\_QUERY, UPDATE\_QUERY, DELETE\_QUERY.
- MEMBERSHIP → pour les actions liées aux memberships : ADD, REMOVE, TRANSFER\_OWNER.

Un audit ne peut jamais cibler plusieurs entités simultanément.

### Dépendances vers les énumérations :

Plusieurs classes du diagramme utilisent des énumérations afin de contraindre les valeurs possibles de certains champs :

- La classe User utilise l'énumération RoleEnum dont les valeurs possibles sont ADMIN et USER.
- La classe Membership utilise l'énumération MembershipRole dont les valeurs possibles sont OWNER et MEMBER.
- La classe Audit utilise l'énumération ActionType dont les valeurs possibles sont CREATE, UPDATE, DELETE, ADD, REMOVE, DEACTIVATE et TRANSFER\_OWNER.
- La classe Deal utilise l'énumération DealType dont les valeurs possibles sont BUY et SELL.
- La classe Deal utilise également l'énumération ProductType dont les valeurs possibles sont GAS, ELECTRICITY, OIL et CO2.

### 3.2.2 Dictionnaire des données

Le dictionnaire des données documente l'ensemble des collections de la base de données MongoDB ainsi que leurs attributs. Chaque champ est décrit avec son type, son caractère obligatoire et une note explicative. Cette documentation garantit une cohérence stricte entre le diagramme de classes, les collections MongoDB et l'application développée.

Tableau 2 : Collection users

| Champ         | Type     | Obligatoire | Description                                   |
|---------------|----------|-------------|-----------------------------------------------|
| _id           | ObjectId | Oui         | Identifiant unique généré par MongoDB         |
| lastname      | String   | Oui         | Nom de famille (ex : Dupont)                  |
| name          | String   | Oui         | Prénom (ex : Jean)                            |
| username      | String   | Oui         | Nom d'utilisateur unique (ex : jdupont)       |
| email         | String   | Oui         | Adresse e-mail unique pour l'authentification |
| phone         | String   | Non         | Numéro de téléphone (ex : +32471234567)       |
| address       | String   | Non         | Adresse postale (ex : Bruxelles)              |
| password_hash | String   | Oui         | Mot de passe hashé via bcrypt                 |
| role          | String   | Oui         | Rôle global : ADMIN ou USER                   |

Cette collection contient les informations relatives aux utilisateurs de l'application.

Tableau 3 : Collection groups

| Champ       | Type     | Obligatoire | Description                                                                |
|-------------|----------|-------------|----------------------------------------------------------------------------|
| _id         | ObjectId | Oui         | Identifiant unique du groupe                                               |
| name        | String   | Oui         | Nom du groupe (ex : Gas Trading Belgium)                                   |
| description | String   | Non         | Description fonctionnelle du groupe                                        |
| status      | Boolean  | Oui         | true = actif, false = désactivé                                            |
| created_at  | Datetime | Oui         | Date et heure de création du groupe, définie automatiquement à la création |

Cette collection représente les groupes collaboratifs de l'application.

Tableau 4 : Collection memberships

| Champ | Type     | Obligatoire | Description                           |
|-------|----------|-------------|---------------------------------------|
| _id   | ObjectId | Oui         | Identifiant unique du membership      |
| role  | String   | Oui         | Rôle dans le groupe : OWNER ou MEMBER |

Cette collection permet de gérer les appartenances des utilisateurs aux groupes.

Tableau 5 : Collection queries

| Champ           | Type     | Obligatoire | Description                                       |
|-----------------|----------|-------------|---------------------------------------------------|
| _id             | ObjectId | Oui         | Identifiant unique de la requête                  |
| query_name      | String   | Oui         | Nom de la requête (ex : DailyConsumptionByZone)   |
| filters         | Dict     | Oui         | Filtres dynamiques JSON (ex : {"product": "GAS"}) |
| selected_fields | Dict     | Non         | Colonnes sélectionnées pour l'affichage           |
| created_at      | Datetime | Oui         | Date de création de la requête                    |

Cette collection contient les requêtes d'analyse créées par les utilisateurs. Les filtres sont stockés sous forme de dictionnaire JSON flexible, permettant des recherches dynamiques sur les deals énergétiques sans modifier la structure de la collection.

Tableau 6 : Collection deals

| Champ         | Type     | Obligatoire | Description                               |
|---------------|----------|-------------|-------------------------------------------|
| _id           | ObjectId | Oui         | Identifiant unique du deal                |
| deal_type     | String   | Oui         | Type de transaction : BUY ou SELL         |
| product       | String   | Oui         | Produit : GAS, ELECTRICITY, OIL ou CO2    |
| delivery_date | Date     | Oui         | Date de livraison ou de règlement du deal |
| amount        | Float    | Oui         | Montant financier total en euros          |
| volume        | Float    | Oui         | Volume négocié en MWh                     |

La collection deals ne contient pas des données créées par l'application. Elle regroupe des données sources brutes, importées telles quelles depuis les systèmes de trading d'ENGIE. L'application n'en est donc ni le producteur ni le propriétaire, elle se contente de les consulter en lecture seule.

C'est pourquoi le diagramme de classes et le dictionnaire des données ne reprennent que les attributs essentiels de l'entité Deal, à savoir le type, le produit, la date de livraison, le montant et le volume. Ces cinq champs suffisent à caractériser un deal dans le modèle de l'application. Tous les autres champs présents dans les documents sont des attributs bruts provenant du système source. Ils ne sont ni définis, ni validés, ni maintenus par l'application, et leur structure reste sous la responsabilité du système qui les fournit.

Il s'agit notamment de champs comme deal\_id, portfolio, counterparty\_name, trader\_code, desk, business\_unit, direction, entity, trade\_date, delivery\_point, delivery\_type, transport\_corridor, price, cash, open\_quantity, margin\_cost, total\_margin\_cost, quantity\_unit, booking\_status et currency. Le modèle souple de MongoDB permet de les conserver dans leur intégralité sans avoir à les intégrer au modèle de l'application, qui ne décrit que les entités qu'elle gère réellement. Concrètement, l'application les utilise seulement de façon dynamique, comme critères de filtrage ou comme colonnes d'affichage des résultats, sans jamais les modéliser.

Tableau 7 : Collection audits

| Champ        | Type     | Obligatoire | Description                                                                     |
|--------------|----------|-------------|---------------------------------------------------------------------------------|
| _id          | ObjectId | Oui         | Identifiant unique de l'entrée d'audit                                          |
| action       | String   | Oui         | Type d'action : CREATE, UPDATE, DELETE, ADD, REMOVE, DEACTIVATE, TRANSFER_OWNER |
| timestamp    | Datetime | Oui         | Date et heure exactes de l'action                                               |
| target_type  | String   | Oui         | Type d'entité ciblée : GROUP, QUERY ou MEMBERSHIP                               |
| target_label | String   | Non         | Nom de l'entité cible (ex : Gas Trading Belgium)                                |
| old_values   | Dict     | Non         | Anciennes valeurs JSON (vide pour une création)                                 |
| new_values   | Dict     | Non         | Nouvelles valeurs JSON (vide pour une suppression)                              |

Cette collection permet d'assurer la traçabilité des opérations réalisées dans l'application.

### 3.2.3 Collections MongoDB

L'application utilise MongoDB comme système de gestion de base de données NoSQL. Ce choix n'est pas arbitraire : l'application d'origine développée au sein d'ENGIE reposait déjà sur une base NoSQL avec MongoDB. J'ai donc fait le choix de conserver la même technologie afin de rester cohérente avec l'existant, de m'inscrire dans la continuité de l'outil utilisé par l'équipe et de faciliter une éventuelle reprise du projet de mon côté.

Contrairement aux bases de données relationnelles traditionnelles qui imposent un schéma fixe, MongoDB repose sur un modèle orienté documents permettant de stocker les données sous forme de documents JSON flexibles. Au-delà de la continuité avec l'application existante, ce choix technologique s'est imposé naturellement dans le cadre du projet pour plusieurs raisons :

- Une grande flexibilité de la structure des données, particulièrement utile pour les filtres dynamiques des queries.
- Une facilité d'évolution du modèle sans nécessiter de migration complexe.
- De bonnes performances pour les recherches dynamiques et les filtres multi-critères.
- Une adaptation aux structures complexes utilisées dans les queries et les audits.
- Une intégration simple et efficace avec FastAPI et Python via la bibliothèque PyMongo.

Les données sont organisées en six collections correspondant aux différentes entités métier de l'application : users, groups, memberships, queries, deals et audits.

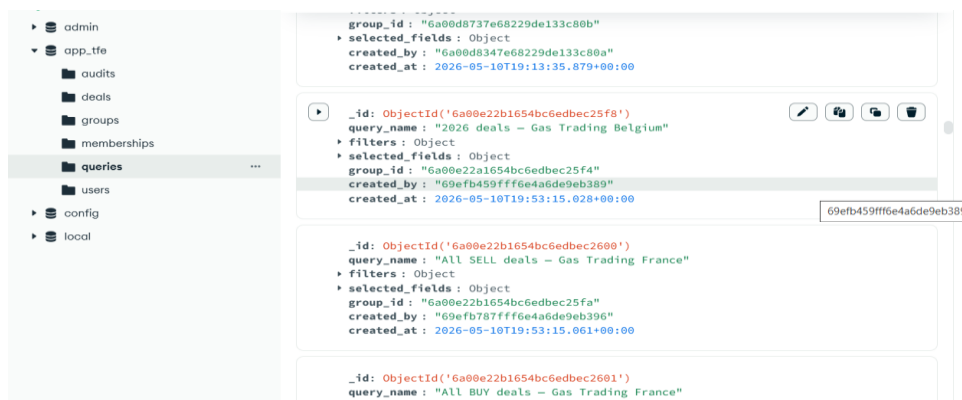


Figure 5 : Organisation des collections MongoDB de l'application représentant les différentes entités métier

**Remarque :** Les champs de référence visibles dans les documents (par exemple owner\_id, group\_id, user\_id, created\_by) ne sont pas repris dans le dictionnaire des données. En NoSQL, ils matérialisent les relations décrites dans le diagramme de classes, et conformément aux bonnes pratiques d'analyse, ce sont les relations qui jouent ce rôle, pas des attributs de clé étrangère.

### Collection Users :

La collection users stocke les comptes de tous les utilisateurs de l'application. Elle est au cœur du système d'authentification et de contrôle des accès. Les mots de passe ne sont jamais stockés en clair, ils sont systématiquement hashés via bcrypt<sup>9</sup> avant enregistrement. Chaque utilisateur possède un rôle global (ADMIN ou USER) qui détermine ses droits sur l'ensemble de la plateforme.



Figure 6 : Document d'un utilisateur stocké dans la collection users de MongoDB

<sup>9</sup>bcrypt : Hachage sécurisé des mots de passe. Disponible sur : <https://pypi.org/project/bcrypt>

### Collection Groups :

La collection groups représente les espaces collaboratifs de l'application. Chaque groupe est identifié par un nom, une description optionnelle, un propriétaire et un statut. La suppression d'un groupe est logique : le champ status passe à false sans supprimer physiquement les données, ce qui permet de conserver l'historique des audits associés.

```

{
  "_id": {
    "$oid": "6a00e22b1654bc6edbec25fa"
  },
  "name": "Gas Trading France",
  "description": "Suivi des transactions de gaz sur le marché français.",
  "owner_id": "69efb787fff6e4a6de9eb396",
  "status": true,
  "created_at": {
    "$date": "2026-05-10T19:53:15.041Z"
  }
}

```

Figure 7 : Document d'un groupe enregistré dans la collection groups de MongoDB

### Collection Memberships :

La collection memberships matérialise la relation entre les utilisateurs et les groupes. Elle permet à un utilisateur d'appartenir à plusieurs groupes simultanément, chacun avec un rôle différent. Chaque document contient une référence vers l'utilisateur, une référence vers le groupe et le rôle attribué dans ce groupe (OWNER ou MEMBER).

```

{
  "_id": {
    "$oid": "6a00e22b1654bc6edbec25f7"
  },
  "group_id": "6a00e22a1654bc6edbec25f4",
  "user_id": "69efb6defff6e4a6de9eb38e",
  "role": "MEMBER"
}

{
  "_id": {
    "$oid": "6a00e22b1654bc6edbec25fb"
  },
  "group_id": "6a00e22b1654bc6edbec25fa",
  "user_id": "69efb787fff6e4a6de9eb396",
  "role": "OWNER"
}

```

Figure 8 : Documents enregistrés dans la collection memberships de MongoDB

## Collection Queries :

La collection queries stocke les requêtes d'analyse créées par les utilisateurs. Chaque query est associée à un groupe, un créateur et un ensemble de filtres dynamiques stockés sous forme de dictionnaire JSON. Cette approche permet une grande souplesse, c'est-à-dire que les filtres peuvent varier d'une query à l'autre sans modifier la structure de la collection. Les champs sélectionnés pour l'affichage des résultats sont également sauvegardés dans le champ selected\_fields.

```
{
  "_id": {
    "$oid": "6a00e22b1654bc6edbec25f8"
  },
  "query_name": "2026 deals – Gas Trading Belgium",
  "filters": {
    "product": "GAS",
    "start_date": "2026-01-01",
    "end_date": "2026-05-01",
    "portfolio": "Gas_NL",
    "deal_type": "BUY"
  },
  "selected_fields": {
    "TradeDate": true,
    "DealId": true,
    "Portfolio": true,
    "Quantity": true,
    "Price": true,
    "DeliveryPoint": true,
    "Entity": true
  },
  "group_id": "6a00e22a1654bc6edbec25f4",
  "created_by": "69efb459fff6e4a6de9eb389",
  "created_at": {
    "$date": "2026-05-10T19:53:15.028Z"
  }
}
```

Figure 9 : Document enregistré dans la collection queries de MongoDB

## Collection Deals :

La collection deals constitue la source de données principale de l'application. Elle contient les opérations de trading énergétique sur lesquelles les queries sont exécutées. Ces données ne sont pas créées par l'application mais elles représentent un référentiel métier réel. Chaque deal contient des informations financières, opérationnelles et logistiques telles que le produit énergétique, le prix, le volume, la contrepartie, le portefeuille, le trader et le statut de réservation.

```
{
  "_id": {},
  "deal_id": "DL-100001",
  "deal_type": "BUY",
  "product": "ELECTRICITY",
  "portfolio": "Elec_NL",
  "desk": "Power Trading",
  "trader_code": "TR007",
  "counterparty_name": "E.ON",
  "business_unit": "Trading",
  "direction": "Short",
  "entity": "ENGIE BE",
  "trade_date": "2027-03-05T00:00:00Z",
  "delivery_date": "2027-03-30T00:00:00Z",
  "volume": 2890,
  "quantity_unit": "MWh",
  "price": 90.49,
  "amount": 261516.1,
  "open_quantity": 2038,
  "cash": 261516.1,
  "margin_cost": 659.33,
  "total_margin_cost": 834.01,
  "delivery_point": "EPEX",
  "delivery_type": "Physical",
  "transport_corridor": "FR-BE",
  "booking_status": "PENDING",
  "currency": "EUR",
  "created_at": {
    "$date": "2026-05-08T10:14:37.305Z"
  }
}
```

Figure 10 : Document enregistré dans la collection deals de MongoDB

## Collection Audits :

La collection audits assure la traçabilité complète de toutes les opérations importantes réalisées dans l'application. Chaque action génère automatiquement un document d'audit contenant l'utilisateur responsable, le type d'action, l'horodatage, l'entité ciblée ainsi que les anciennes et nouvelles valeurs lorsque cela est pertinent. Les entrées d'audit sont en lecture seule et ne peuvent jamais être modifiées, garantissant ainsi l'intégrité de l'historique. Les types d'actions tracées incluent notamment la création et la modification de groupes et de queries, l'ajout et la suppression de membres, le transfert de propriété et la désactivation de groupes.

```
{
  "_id": { },
  "action": "DEACTIVATE",
  "timestamp": {
    "$date": "2026-04-27T19:58:49.022Z"
  },
  "target_type": "GROUP",
  "target_id": "69efbfeeb045d8fa7c066f84",
  "target_label": "test ",
  "user_id": "69d4db88962fcee86f06f73d",
  "group_id": "69efbfeeb045d8fa7c066f84",
  "old_values": {
    "status": true
  },
  "new_values": {
    "status": false
  }
}
```

Figure 11 : Document enregistré dans la collection audits de MongoDB

## 3.3 Autres diagrammes

Afin de compléter l'analyse fonctionnelle et technique de l'application, plusieurs diagrammes UML supplémentaires ont été réalisés. Ils permettent de visualiser les processus métier clés et les interactions entre les différents composants de l'application.

### 3.3.1 Diagramme de séquence d'Authentification JWT

Ce diagramme illustre le flux complet d'authentification d'un utilisateur dans l'application. Il met en évidence les échanges entre le frontend Angular, le backend FastAPI et la base de données MongoDB lors de la connexion.

Le processus se déroule en plusieurs étapes :

- L'utilisateur saisit son adresse e-mail et son mot de passe dans le formulaire de connexion.
- Le frontend envoie une requête POST vers l'endpoint /auth/login du backend.
- Le backend vérifie les identifiants dans la collection users et compare le mot de passe avec le hash bcrypt stocké.
- Si les identifiants sont corrects, un token JWT signé est généré et renvoyé au frontend.
- Le frontend stocke le token dans le localStorage et redirige l'utilisateur vers le dashboard.
- Pour chaque requête suivante, le token est inclus dans le header Authorization.



- Si le token est invalide ou expiré, l'accès est refusé et l'utilisateur est redirigé vers la page de connexion.

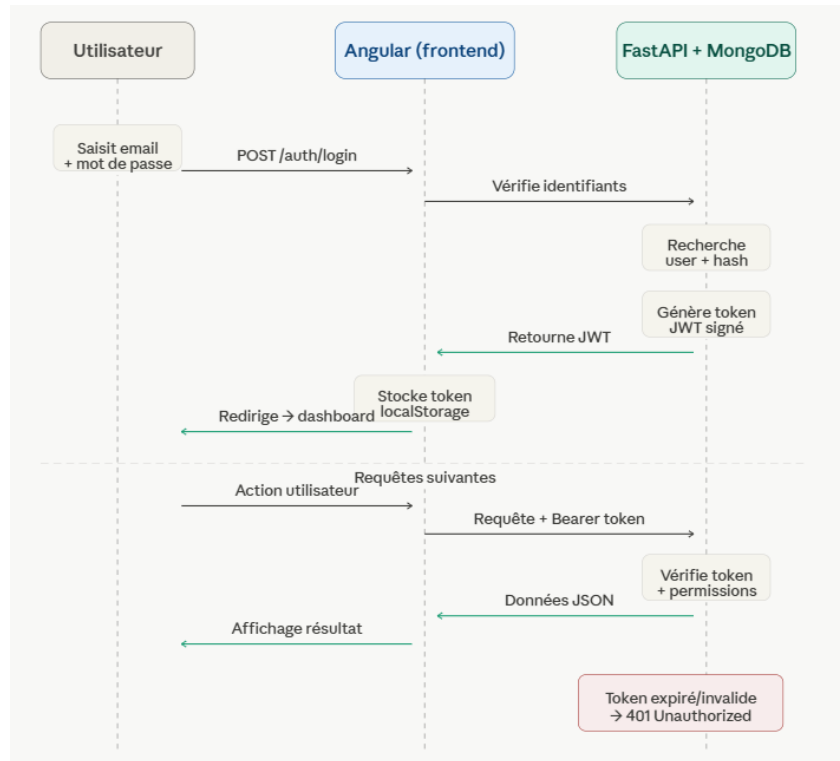


Figure 12 : Diagramme de séquence illustrant le processus d'authentification sécurisé de l'application

#### Description du processus :

1. Saisie des identifiants : L'utilisateur saisit son adresse e-mail et son mot de passe dans le formulaire de connexion du frontend Angular.
2. Envoi de la requête : Le frontend transmet les identifiants au backend FastAPI via une requête HTTP POST vers l'endpoint /auth/login.
3. Vérification en base de données : Le backend recherche l'utilisateur dans la collection users de MongoDB et compare le mot de passe fourni avec le hash bcrypt stocké.
4. Génération du token JWT : Si les identifiants sont corrects, le backend génère un token JWT signé contenant l'identifiant de l'utilisateur et une durée de validité. Ce token est renvoyé au frontend.
5. Stockage du token : Le frontend stocke le token dans le localStorage du navigateur et redirige automatiquement l'utilisateur vers le dashboard.
6. Requêtes suivantes : Pour chaque action suivante, le frontend inclut le token JWT dans le header Authorization de chaque requête HTTP.
7. Vérification du token : Le backend vérifie la validité du token et les permissions de l'utilisateur avant d'autoriser l'accès à l'endpoint demandé.

8. Token expiré ou invalide : Si le token est invalide ou expiré, le backend retourne une erreur 401 Unauthorized et l'utilisateur est automatiquement redirigé vers la page de connexion.

### 3.3.2 Diagramme de séquence de Reporting automatique en PDF

Ce diagramme illustre le mécanisme de génération automatique des rapports PDF et d'envoi des alertes par email. Il implique quatre composants : APScheduler<sup>10</sup>, FastAPI, MongoDB et le serveur SMTP.

Deux scénarios sont représentés :

#### Rapport mensuel automatique

- Le dernier jour de chaque mois, le scheduler APScheduler déclenche automatiquement le job de reporting.
- Le backend interroge la collection audits pour récupérer l'ensemble des actions du mois groupées par groupe.
- Les emails des propriétaires de chaque groupe sont récupérés dans la collection users.
- Un rapport PDF est généré via la bibliothèque ReportLab.
- Le rapport est envoyé par email au propriétaire du groupe via le serveur SMTP.

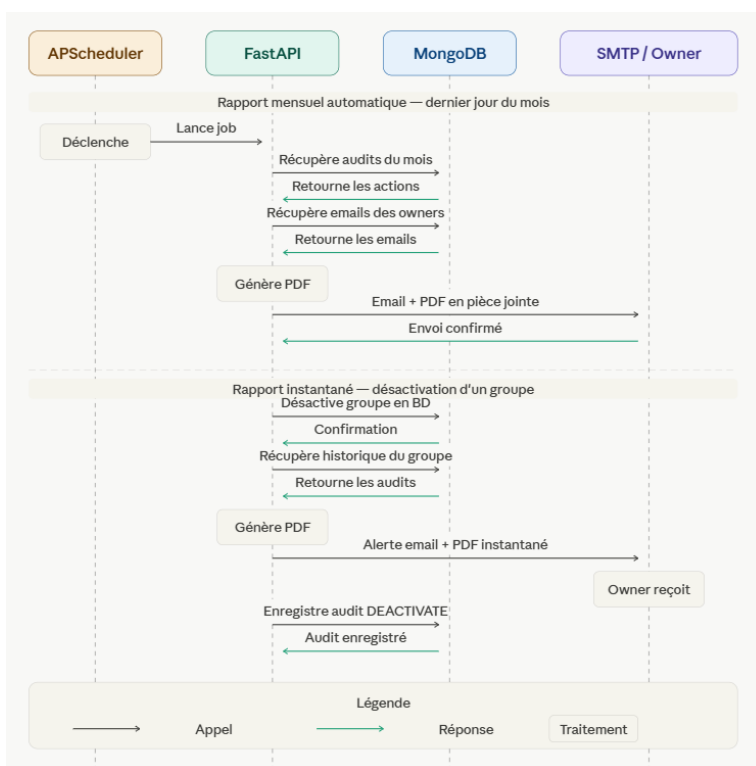


Figure 13 : Diagramme de séquence illustrant le processus de génération automatique des rapports d'audit PDF

#### Rapport instantané lors d'une désactivation

<sup>10</sup>APScheduler : Documentation officielle. Disponible sur : <https://apscheduler.readthedocs.io>

- Lors de la désactivation d'un groupe, le backend récupère immédiatement l'historique des audits du groupe.
- Un rapport PDF instantané est généré et envoyé par email au propriétaire du groupe.
- L'action de désactivation est enregistrée dans la collection audits.

### 3.3.3 Diagramme d'activité

Ce diagramme illustre l'enchaînement des actions, des décisions et des validations lors des principales opérations de l'application. Trois workflows importants sont représentés.

#### Création d'un groupe

- L'utilisateur authentifié remplit le formulaire de création de groupe.
- Le backend vérifie les permissions de l'utilisateur.
- Le groupe est enregistré dans la collection groups.
- Le créateur est automatiquement désigné comme propriétaire via un membership de rôle OWNER.
- Un audit de type CREATE est généré automatiquement.

#### Ajout d'un membre

- Le propriétaire sélectionne un utilisateur à ajouter au groupe.
- Le backend vérifie que l'utilisateur n'est pas déjà membre du groupe.
- Le membership est créé dans la collection memberships.
- Un audit de type ADD est généré automatiquement.

#### Exécution d'une query

- L'utilisateur sélectionne une query existante et clique sur Exécuter.
- Le backend récupère les filtres stockés dans la collection queries.
- Une requête dynamique est construite et exécutée sur la collection deals.
- Les résultats sont renvoyés au frontend et affichés sous forme de tableau.

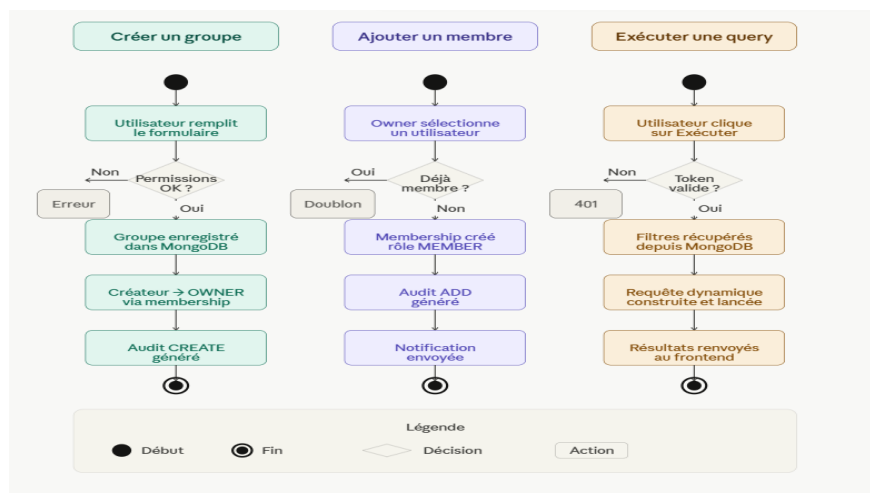


Figure 14 : Diagrammes d'activités illustrant les principaux processus métier de l'application

### 3.3.4 BPMN

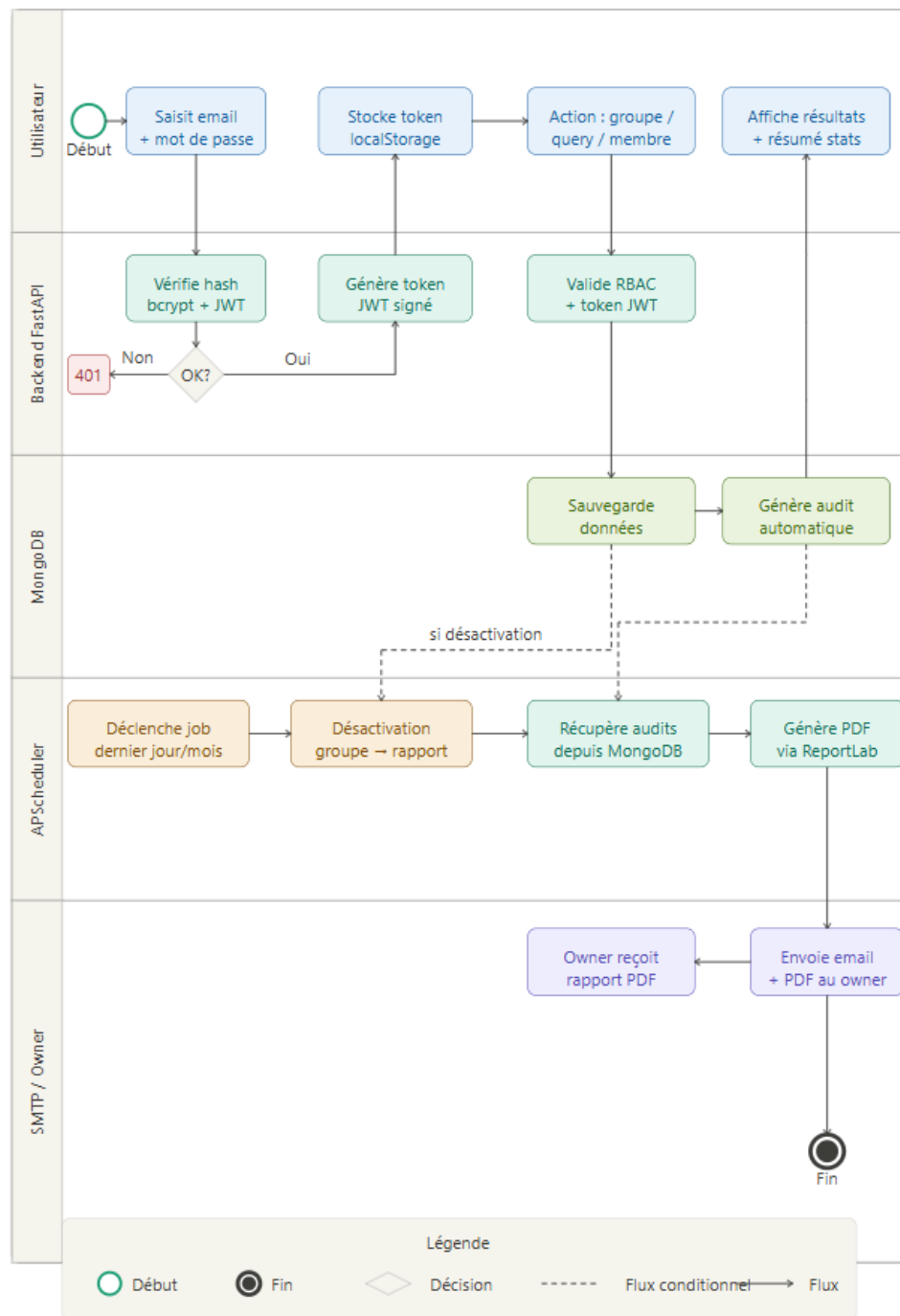


Figure 15 : Diagramme BPMN illustrant le fonctionnement global de l'application

## 4. DOSSIER TECHNIQUE

### 4.1 Technologies utilisées

#### 4.1.1 Vue d'ensemble de l'architecture

L'application repose sur une architecture moderne de type client-serveur organisée en trois couches distinctes et complémentaires. Cette séparation claire des responsabilités garantit une meilleure maintenabilité, une sécurité renforcée et une évolutivité facilitée.

- Le frontend Angular constitue l'interface utilisateur. Il gère l'affichage des données, les formulaires, la navigation et les échanges avec le backend via des requêtes HTTP sécurisées.
- Le backend FastAPI constitue le cœur logique de l'application. Il gère les règles métier, l'authentification JWT, les permissions RBAC, les audits et les opérations CRUD.
- MongoDB assure la persistance des données dans les six collections de l'application.

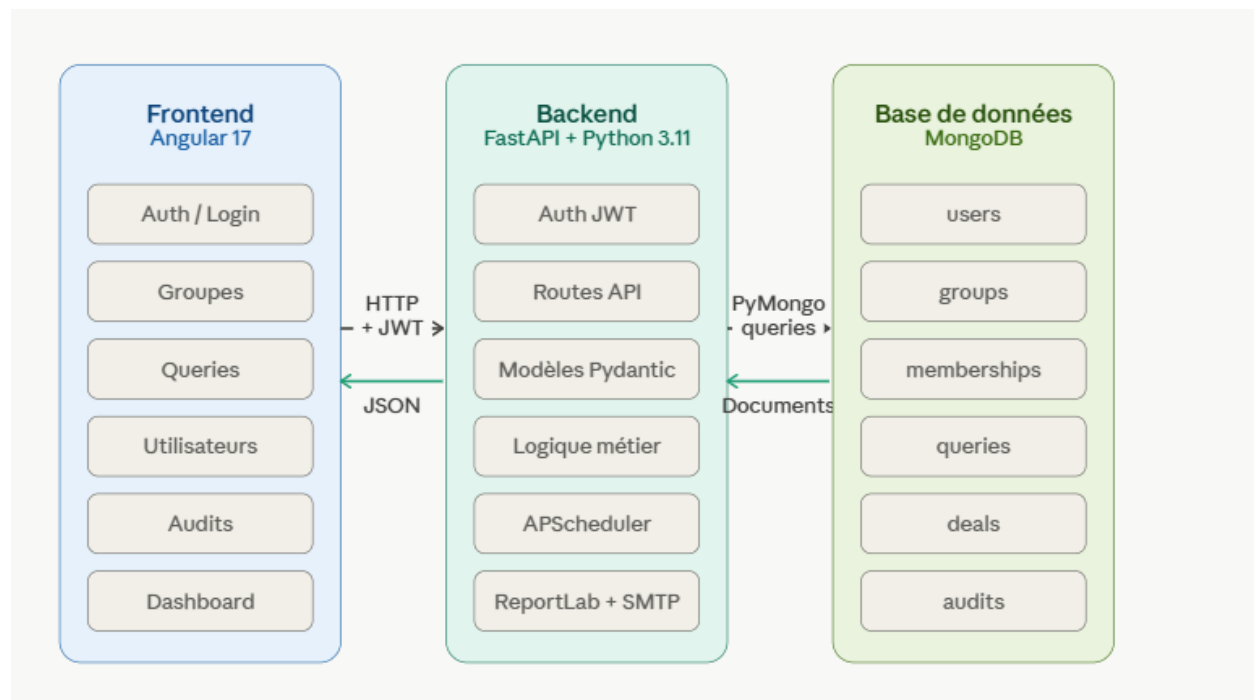


Figure 16 : Schéma d'architecture de l'application

#### 4.1.2 Frontend Angular

Le frontend de l'application a été développé avec Angular 17 en TypeScript. Angular a été choisi pour plusieurs raisons importantes :

- Une architecture modulaire permettant une organisation claire du code.
- Une séparation nette des composants facilitant la maintenance.
- Une gestion avancée du routing pour la navigation entre les pages.
- De bonnes performances grâce au mécanisme de détection des changements.
- Une intégration simple et efficace avec les APIs REST via HttpClient.

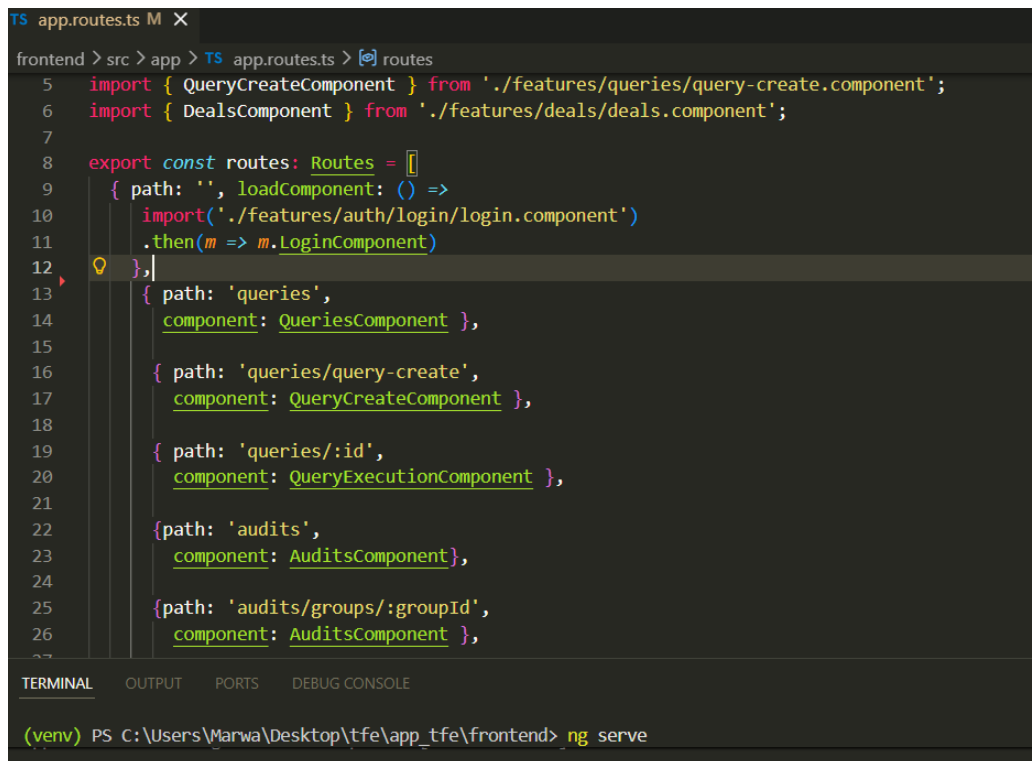
## Angular Standalone Components

Le projet utilise les composants standalone d'Angular, introduits dans les versions récentes du framework. Cette approche supprime la nécessité des modules NgModule et permet une organisation plus claire et plus modulaire du code. Chaque composant est autonome et déclare directement ses dépendances, ce qui simplifie la structure globale du projet et facilite la maintenance.

## Routing Angular

Le routing Angular gère la navigation entre les différentes pages de l'application. Les principales routes définies concernent :

- L'authentification et la page de connexion.
- Le dashboard principal.
- La gestion des utilisateurs.
- La gestion des groupes et des memberships.
- La création et l'exécution des queries.
- La consultation des audits.



```
TS app.routes.ts M X
frontend > src > app > TS app.routes.ts > routes
5 import { QueryCreateComponent } from './features/queries/query-create.component';
6 import { DealsComponent } from './features/deals/deals.component';
7
8 export const routes: Routes = [
9   { path: '', loadComponent: () =>
10     import('./features/auth/login/login.component')
11     .then(m => m.LoginComponent)
12   },
13   { path: 'queries',
14     component: QueriesComponent },
15   { path: 'queries/query-create',
16     component: QueryCreateComponent },
17   { path: 'queries/:id',
18     component: QueryExecutionComponent },
19   { path: 'audits',
20     component: AuditsComponent },
21   { path: 'audits/groups/:groupId',
22     component: AuditsComponent },
23 ]
24
25
26
27
TERMINAL OUTPUT PORTS DEBUG CONSOLE
(venv) PS C:\Users\Marwa\Desktop\tfe\app_tfe\frontend> ng serve
```

Figure 17 : Capture du fichier app.routes.ts illustrant la configuration des routes Angular de l'application

## Services Angular

Les services Angular centralisent les appels HTTP vers le backend ainsi que certaines logiques métier frontend. Les requêtes HTTP sont effectuées à l'aide du module HttpClient intégré dans Angular. Cette

approche permet de séparer la logique de communication du reste des composants et de faciliter la réutilisation du code.

### Gestion des tokens JWT

Après authentification, le frontend stocke le token JWT dans le localStorage du navigateur. Ce token est ensuite inclus automatiquement dans le header Authorization de chaque requête HTTP sécurisée vers le backend :

#### Authorization: Bearer <token>

Cette approche permet de sécuriser l'accès aux endpoints protégés du backend sans nécessiter de gestion de session côté serveur. Lors de la déconnexion, le token est supprimé du localStorage afin d'empêcher toute utilisation future.

### 4.1.3 Backend FastAPI

Le backend de l'application a été développé avec FastAPI en Python 3.11. FastAPI est un framework moderne permettant de développer rapidement des APIs REST performantes et sécurisées. Il présente plusieurs avantages importants dans le cadre de ce projet :

- Une rapidité de développement grâce à sa syntaxe claire et moderne.
- Une génération automatique de documentation interactive via Swagger UI.
- Une validation automatique des données entrantes via Pydantic.
- De bonnes performances adaptées aux besoins de l'application.
- Une compatibilité native avec Python et une intégration simple avec MongoDB.

#### Structure du backend :

Le backend est organisé en plusieurs couches afin de garantir une séparation claire des responsabilités :

- routes/ : définition des endpoints REST pour chaque fonctionnalité.
- models/ : modèles Pydantic pour la validation des données.
- services/ : logique métier et services techniques (scheduler, email, PDF).
- config.py : configuration centralisée (MongoDB, SMTP, JWT).
- main.py : point d'entrée de l'application.

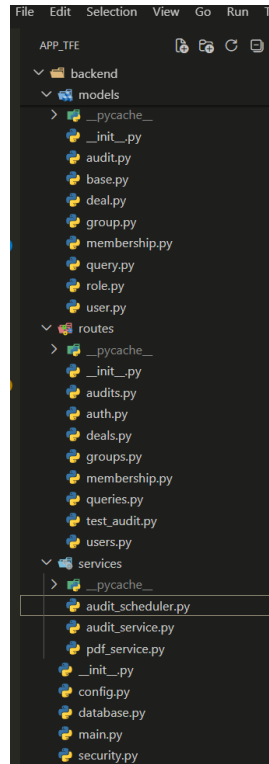


Figure 18 : Capture de l'organisation du backend FastAPI

### APIs REST :

Le backend expose plusieurs endpoints REST permettant de gérer l'ensemble des fonctionnalités de l'application :

- La gestion des utilisateurs (création, modification, suppression, consultation).
- La gestion des groupes et des memberships.
- La création, modification et exécution des queries.
- Les recherches dynamiques sur les deals énergétiques.
- La consultation des audits et de l'historique des actions.

Les échanges entre le frontend et le backend sont réalisés au format JSON. Chaque requête inclut le token JWT dans le header Authorization afin de garantir la sécurité des échanges.

### Validation des données :

FastAPI utilise Pydantic<sup>11</sup> afin de valider automatiquement les données reçues dans les requêtes API. Chaque modèle définit les champs attendus, leurs types et leurs contraintes. Cette validation permet de sécuriser les entrées utilisateur, d'éviter certaines erreurs de traitement et de garantir la cohérence des données avant leur enregistrement dans MongoDB.

<sup>11</sup>Pydantic : Documentation officielle. Disponible sur : <https://docs.pydantic.dev>



## Bibliothèques principales :

Les bibliothèques suivantes ont été utilisées dans le backend :

- PyMongo : interactions avec MongoDB.
- Pydantic : validation automatique des données entrantes.
- python-jose<sup>12</sup> : génération et vérification des tokens JWT.
- passlib<sup>13</sup> / bcrypt : hachage sécurisé des mots de passe.
- APScheduler : planification des tâches automatiques.
- ReportLab : génération des rapports PDF.
- smtplib : envoi des emails via SMTP.

### 4.1.4 Base de données MongoDB

L'application utilise MongoDB comme base de données NoSQL. Ce choix est justifié par la flexibilité du modèle orienté documents, particulièrement adaptée aux filtres dynamiques des queries et aux structures évolutives du projet. L'intégration avec FastAPI est assurée via la bibliothèque PyMongo<sup>14</sup>, qui permet une manipulation simple et efficace des documents JSON côté backend. La structure détaillée des collections est présentée dans la section Persistance des données du dossier d'analyse.

### 4.1.5 Sécurité JWT/RBAC

La sécurité constitue un élément central du projet en raison de la sensibilité des données manipulées dans le contexte du trading énergétique. Plusieurs mécanismes complémentaires ont été implémentés afin de garantir un contrôle d'accès strict et une protection efficace des données.

- Authentification JWT

Le système d'authentification repose sur les JSON Web Tokens (JWT). Après connexion, le backend génère un token signé contenant l'identifiant de l'utilisateur et une durée de validité. Le rôle de l'utilisateur est ensuite vérifié côté serveur, à partir de la base de données, à chaque requête nécessitant un contrôle de permissions. Le frontend stocke ce token dans le localStorage et l'inclut dans le header Authorization de chaque requête HTTP suivante. Cette approche permet d'authentifier les utilisateurs, de sécuriser les échanges et de contrôler les accès sans nécessiter de gestion de session côté serveur.

- Gestion des permissions (RBAC)

L'application utilise un système RBAC (Role-Based Access Control) permettant de gérer finement les droits d'accès selon le profil de chaque utilisateur. Trois niveaux de permissions ont été définis :

- ADMIN : accès global à toutes les fonctionnalités de la plateforme.
- OWNER : droits de gestion sur son groupe, ses membres et ses queries.
- MEMBER : droits limités à la consultation et à l'exécution des queries du groupe.
- Protection des endpoints

---

<sup>12</sup>python-jose : Documentation officielle. Disponible sur : <https://python-jose.readthedocs.io>

<sup>13</sup>Passlib : Documentation officielle. Disponible sur : <https://passlib.readthedocs.io>

<sup>14</sup>PyMongo : Documentation officielle. Disponible sur : <https://pymongo.readthedocs.io>

Les endpoints sensibles du backend sont protégés via une dépendance FastAPI qui vérifie automatiquement la présence et la validité du token JWT avant chaque traitement. Des contrôles supplémentaires vérifient les permissions de l'utilisateur connecté selon son rôle global et son rôle dans le groupe concerné.

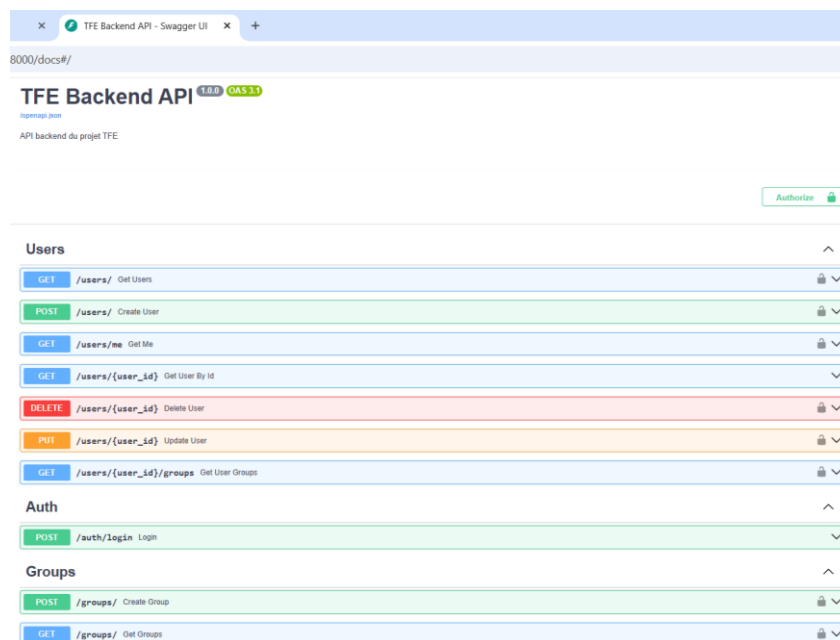


Figure 19 : Capture de la documentation Swagger des endpoints sécurisés du backend FastAPI

- Sécurisation des mots de passe

Les mots de passe ne sont jamais stockés en clair dans la base de données. Ils sont systématiquement hashés via bcrypt avant enregistrement, garantissant ainsi la sécurité des comptes utilisateurs même en cas de compromission de la base de données.

#### 4.1.6 Outils de développement

Plusieurs outils professionnels ont été utilisés tout au long du développement du projet afin d'assurer une organisation rigoureuse et une qualité de code élevée.

Tableau 8 : Outils utilisés durant le développement du projet

| Outil                 | Utilisation                 |
|-----------------------|-----------------------------|
| Visual Studio Code    | Développement principal     |
| Git / GitHub          | Gestion de versions         |
| Swagger               | Documentation API           |
| Postman               | Test API                    |
| MongoDB Compass       | Gestion des collections     |
| Microsoft Teams       | Communication projet        |
| Draw.io <sup>15</sup> | Modélisation des diagrammes |

<sup>15</sup>Draw.io : Outil de modélisation UML. Disponible sur : <https://app.diagrams.net>

## Gestion du code source avec Git et GitHub

Tout au long du développement, le code source de l'application a été versionné à l'aide de Git et hébergé sur GitHub. Cette approche a permis de structurer le développement de façon rigoureuse et professionnelle.

Le projet a été organisé selon les pratiques suivantes :

- Création de branches dédiées pour chaque fonctionnalité développée (feature-backend, feature-groups, feature-queries, feature-frontend, feature/audit-reporting).
- Réalisation de commits réguliers avec des messages descriptifs précisant la nature des modifications apportées.
- Fusion des branches via des pull requests après validation du code.
- Conservation d'un historique complet et traçable de toutes les modifications réalisées.

Cette organisation a permis de travailler de façon structurée, d'éviter les conflits de code et de maintenir une cohérence globale du projet tout au long de son développement.

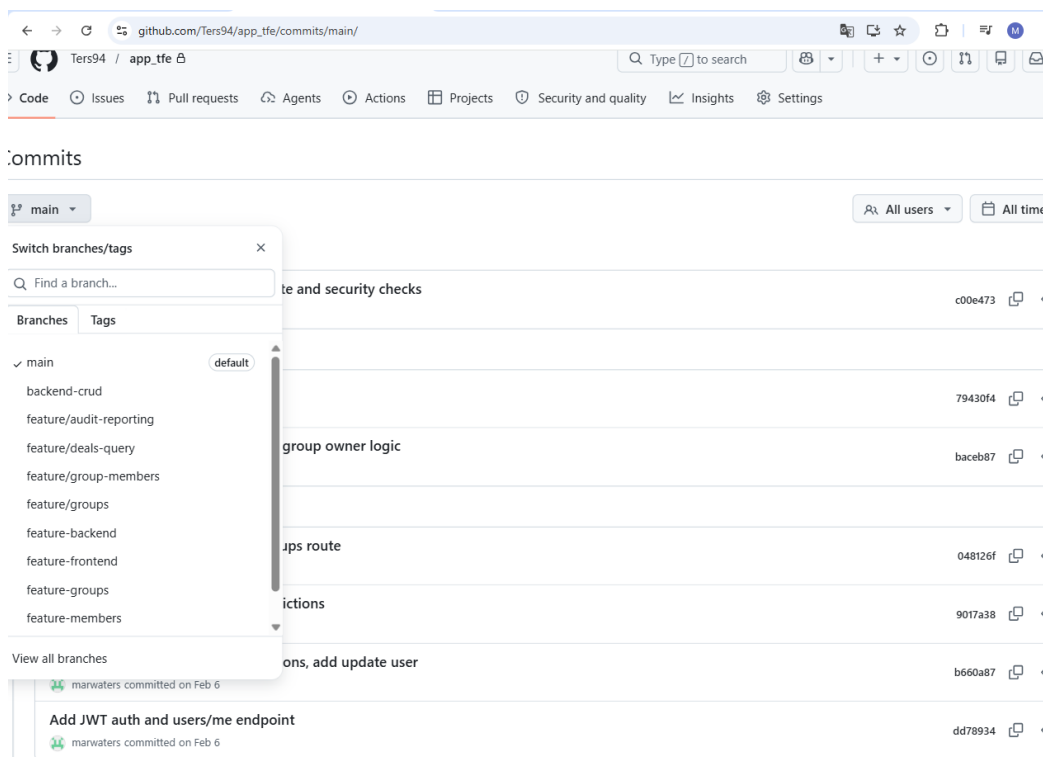


Figure 20 : Organisation du projet sur GitHub avec gestion des branches et historique des commits.

## 4.2 Automatisation : Reporting et alertes

L'un des mécanismes techniques les plus importants du projet est le système de reporting automatique. Il repose sur APScheduler, un planificateur de tâches Python intégré directement dans le backend FastAPI. Ce système permet de déclencher automatiquement des traitements à des moments précis, sans aucune intervention manuelle de l'utilisateur.

#### 4.2.1 Rapport mensuel automatique

Le dernier jour de chaque mois, le scheduler déclenche automatiquement le job de reporting. Le processus se déroule en plusieurs étapes :

1. APScheduler déclenche le job de reporting automatiquement.
2. Le backend interroge la collection audits afin de récupérer toutes les actions réalisées durant le mois en cours, groupées par groupe.
3. Les adresses e-mail des propriétaires de chaque groupe sont récupérées dans la collection users.
4. Un rapport PDF est généré pour chaque groupe via la bibliothèque ReportLab. Il contient la liste des actions réalisées avec pour chaque action l'utilisateur responsable, le type d'action et l'horodatage.
5. Le rapport est envoyé par e-mail au propriétaire du groupe via le serveur SMTP configuré dans les variables d'environnement.

#### 4.2.2 Rapport instantané lors d'une désactivation

Lors de la désactivation d'un groupe, un rapport instantané est généré immédiatement sans attendre la fin du mois. Le processus est le suivant :

1. L'utilisateur désactive un groupe via l'interface.
2. Le backend désactive le groupe dans MongoDB et enregistre un audit de type DEACTIVATE.
3. L'historique complet des audits du groupe est récupéré depuis MongoDB.
4. Un rapport PDF instantané est généré via ReportLab.
5. Le rapport est envoyé immédiatement par e-mail au propriétaire du groupe pour l'informer de l'action réalisée.

#### 4.2.3 Configuration SMTP

Les paramètres de connexion au serveur SMTP, à savoir le serveur, port, adresse e-mail et mot de passe, sont stockés dans les variables d'environnement de l'application. Cette approche garantit la confidentialité des informations de connexion et facilite la configuration selon l'environnement d'exécution.

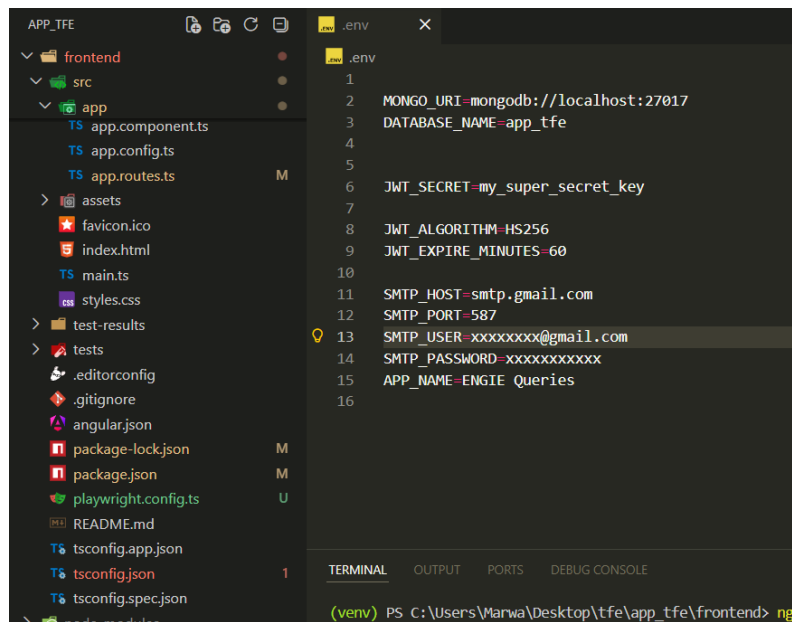


Figure 21 : Capture de la configuration SMTP

## 4.3 Mécanismes techniques clés

Cette section met en évidence certains mécanismes techniques importants du projet, conformément aux bonnes pratiques de développement backend et NoSQL.

### 4.3.1 Exécution dynamique des queries

L'exécution d'une query constitue le mécanisme central de l'application. Lorsqu'un utilisateur clique sur Exécuter, le processus suivant est déclenché :

1. Le frontend envoie une requête GET vers l'endpoint `/queries/{id}/execute` avec le token JWT dans le header Authorization.
2. Le backend récupère les filtres stockés dans la collection queries via PyMongo.
3. Une requête MongoDB est construite dynamiquement à partir des filtres définis par l'utilisateur.
4. La requête est exécutée sur la collection deals et les résultats correspondants sont récupérés.
5. Les résultats sont triés par date de trade puis renvoyés au frontend Angular sous forme de JSON.
6. Le frontend affiche les résultats dans un tableau dynamique avec les colonnes sélectionnées.

Cette approche permet des recherches flexibles et performantes sans nécessiter de modification du code lors de l'ajout de nouveaux critères de filtrage.

### 4.3.2 Filtres MongoDB et récupération des données

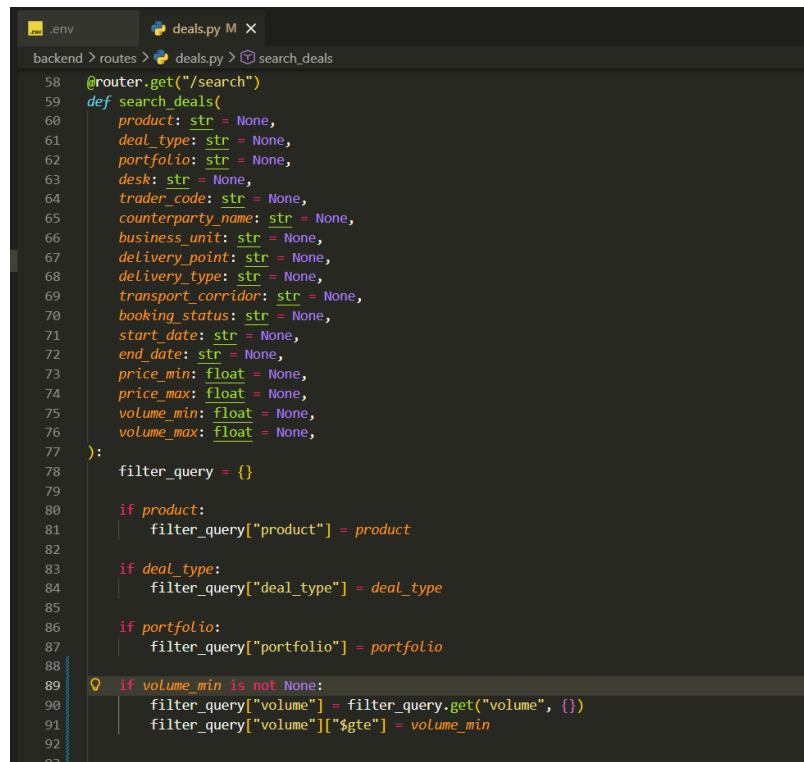
Les recherches sur les deals énergétiques utilisent les opérateurs natifs de MongoDB afin de construire des requêtes multi-critères performantes. Les principaux opérateurs utilisés sont les suivants :

- `$match` : filtre les documents selon les critères définis.
- `$gte` et `$lte` : filtrage par plage de valeurs pour les prix, les volumes et les dates.

- Égalité exacte : filtrage sur les champs catégoriels comme le produit, le type de deal, le portefeuille, la contrepartie ou le trader.

Les filtres peuvent être combinés afin de construire des recherches complexes. Par exemple, il est possible de rechercher tous les deals de type GAS, avec un prix supérieur à 40, sur un portefeuille spécifique et entre deux dates données.

Aucune donnée n'est stockée côté frontend, les résultats sont récupérés en temps réel depuis MongoDB à chaque exécution, garantissant ainsi que les données affichées sont toujours à jour.



```

58 @router.get("/search")
59 def search_deals(
60     product: str = None,
61     deal_type: str = None,
62     portfolio: str = None,
63     desk: str = None,
64     trader_code: str = None,
65     counterparty_name: str = None,
66     business_unit: str = None,
67     delivery_point: str = None,
68     delivery_type: str = None,
69     transport_corridor: str = None,
70     booking_status: str = None,
71     start_date: str = None,
72     end_date: str = None,
73     price_min: float = None,
74     price_max: float = None,
75     volume_min: float = None,
76     volume_max: float = None,
77 ):
78     filter_query = {}
79
80     if product:
81         filter_query["product"] = product
82
83     if deal_type:
84         filter_query["deal_type"] = deal_type
85
86     if portfolio:
87         filter_query["portfolio"] = portfolio
88
89     if volume_min is not None:
90         filter_query["volume"] = filter_query.get("volume", {})
91         filter_query["volume"]["$gte"] = volume_min
92
93

```

Figure 22 : Capture des filtres dynamiques MongoDB pour la recherche des deals énergétiques

#### 4.3.3 Génération automatique des audits

La génération des audits est directement intégrée dans la logique métier du backend FastAPI. Chaque fois qu'une opération sensible est exécutée, le processus suivant est déclenché automatiquement :

1. Le backend valide les permissions de l'utilisateur connecté.
2. L'opération métier est réalisée (création, modification, suppression).
3. Un document d'audit est créé automatiquement avec les informations suivantes : utilisateur responsable, type d'action, horodatage, entité ciblée, anciennes et nouvelles valeurs.
4. Le document est enregistré dans la collection audits de MongoDB.

Les types d'actions tracées sont les suivants :

- CREATE, UPDATE, DELETE : pour les groupes et les queries.
- ADD, REMOVE, TRANSFER\_OWNER : pour les memberships.
- DEACTIVATE : pour la désactivation d'un groupe.

Cette approche garantit une traçabilité systématique sans aucune intervention manuelle. Le système d'audit fonctionne de manière transparente et ne nécessite aucune action supplémentaire de la part des utilisateurs.

```

d > routes > groups.py > deactivate_group
def deactivate_group(
    group = db.groups.find_one({"_id": ObjectId(group_id)})

    if not group:
        raise HTTPException(status_code=404, detail="Group not found")

    if group["owner_id"] != current_user and not is_admin(current_user):
        raise HTTPException(status_code=403, detail="Not authorized")

    db.groups.update_one(
        {"_id": ObjectId(group_id)},
        {"$set": {"status": False}}
    )

    create_audit(
        action="DEACTIVATE",
        current_user=current_user,
        group_id=group_id,
        target_label=group.get("name"),
        old_values={"status": group.get("status", True)},
        new_values={"status": False}
    )

    try:
        owner = db.users.find_one({"_id": ObjectId(group["owner_id"])})

        if owner and owner.get("email"):
            group_name = group.get("name", "")
            owner_email = owner.get("email")
            owner_name = f"{owner.get('name', '')} {owner.get('lastname', '')}.strip() or owner.get('username', '')"
            now = datetime.utcnow()
            month_label = f"Suppression du groupe - {now.strftime('%d/%m/%Y')}"

            audit_docs = list(db.audits.find(
                {"group_id": group_id}
            ).sort("timestamp", -1))

```

Figure 23 : Capture de l'implémentation de la génération automatique des audits dans le backend FastAPI

#### 4.3.4 Historisation des anciennes et nouvelles valeurs

Pour les opérations de modification, le système d'audit enregistre non seulement l'action réalisée mais également les anciennes et les nouvelles valeurs des champs modifiés au format JSON.

Par exemple, si le nom d'un groupe est modifié, l'audit contiendra :

old\_values: {"name": "Gas Trading"} et

new\_values: {"name": "Gas Trading Belgium"}.

Cette fonctionnalité permet d'identifier précisément chaque changement réalisé et de reconstituer l'historique complet des modifications.

#### 4.3.5 Sauvegarde des filtres en JSON

Les queries ne stockent pas de données puisqu'elles sauvegardent uniquement des filtres sous forme de dictionnaire JSON flexible dans MongoDB. Cette approche permet à chaque utilisateur de définir ses propres critères d'analyse et de les réutiliser à tout moment sans avoir à les reconfigurer. Les filtres sont appliqués dynamiquement lors de chaque exécution, garantissant des résultats toujours à jour.

## 4.4 Gestion des rôles et permissions

La gestion des rôles constitue un élément fondamental de l'application. Les permissions ne dépendent pas uniquement du rôle global de l'utilisateur mais elles varient également selon son rôle au sein du groupe concerné et selon le type d'opération effectuée. Cette double logique de permissions garantit une organisation cohérente et sécurisée des accès.

### 4.4.1 Rôle Administrateur

L'administrateur possède un accès global à l'ensemble des fonctionnalités de l'application. Ce rôle a été introduit spécifiquement dans le cadre de ce projet afin de structurer les droits d'accès de façon plus claire et plus sécurisée. Il peut notamment :

- Créer, modifier et supprimer des comptes utilisateurs.
- Consulter et gérer l'ensemble des groupes de la plateforme, indépendamment de leur propriétaire.
- Superviser toutes les queries et accéder à tous les audits.
- Intervenir sur toutes les ressources du système en cas de besoin.

### 4.4.2 Rôle Owner

L'owner correspond au propriétaire d'un groupe. Lorsqu'un utilisateur crée un groupe, il en devient automatiquement le propriétaire via la création d'un membership de rôle OWNER. Il dispose de droits étendus sur son espace collaboratif :

- Modifier les informations du groupe.
- Ajouter ou supprimer des membres.
- Transférer la propriété du groupe à un autre membre.
- Gérer les queries associées au groupe.
- Consulter l'historique complet des audits du groupe.
- Recevoir les rapports PDF d'audit mensuels et les alertes instantanées en cas de désactivation du groupe.

### 4.4.3 Rôle Member

Le member est un utilisateur appartenant à un groupe sans en être le propriétaire. Il dispose de permissions plus limitées mais peut néanmoins participer activement aux activités collaboratives :

- Consulter les informations et les membres du groupe.
- Créer et modifier des queries associées au groupe.
- Exécuter des recherches sur les deals énergétiques.
- Consulter les audits du groupe dont il est membre.

### 4.4.4 Gestion des permissions dans les groupes

Les permissions ne dépendent pas uniquement du rôle global de l'utilisateur. Certaines actions sont conditionnées par le rôle dans le groupe concerné. Voici les principales règles appliquées :



- Seul le propriétaire peut désactiver un groupe.
- Seul le propriétaire peut ajouter ou supprimer des membres.
- Seul le propriétaire peut transférer la propriété du groupe.
- Les membres et le propriétaire peuvent créer et modifier des queries.
- L'administrateur dispose de tous les droits sur tous les groupes, quelle que soit sa position dans le groupe.

#### 4.4.5 Sécurité des échanges frontend/backend

Les échanges entre Angular et FastAPI sont réalisés via des requêtes HTTP sécurisées. Chaque requête contient le token JWT dans le header Authorization ainsi que les données JSON nécessaires à l'opération. Le backend valide systématiquement les éléments suivants avant d'exécuter tout traitement :

- La présence et la validité du token JWT.
- L'identité de l'utilisateur connecté.
- Son rôle global dans l'application.
- Son rôle dans le groupe concerné par l'opération.
- Ses droits sur la ressource ciblée.

Cette double vérification, côté frontend dans le `ngOnInit()` et côté backend dans les dépendances FastAPI, garantit qu'aucune opération non autorisée ne peut être exécutée, même en cas de contournement de l'interface utilisateur.

#### 4.4.6 Protection des routes Angular

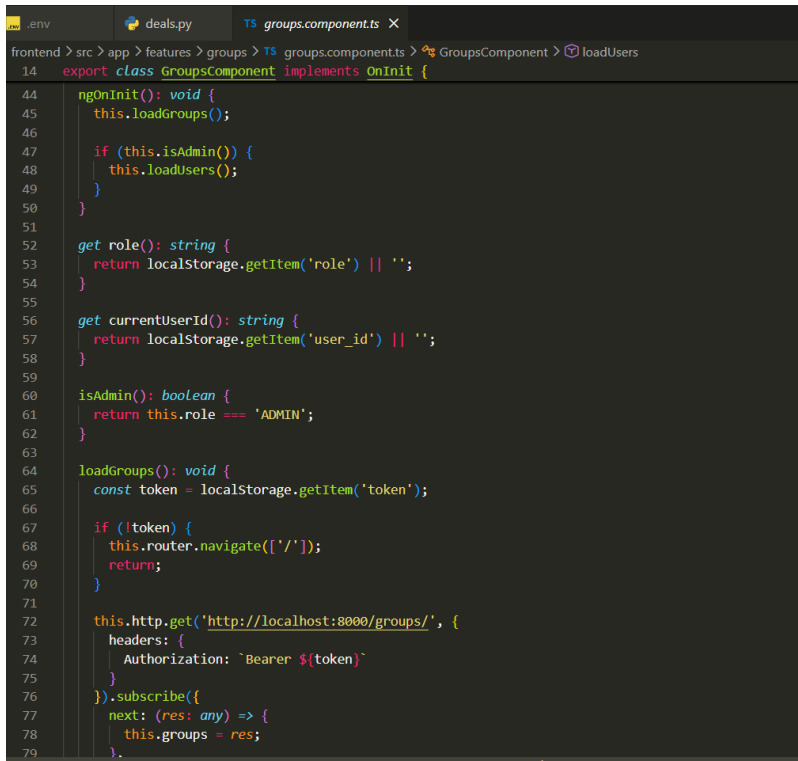
La protection des routes dans le frontend Angular a été implémentée directement au sein des composants de l'application. Contrairement à l'utilisation de Guards Angular formels, les vérifications d'accès sont réalisées dans la méthode `ngOnInit()` de chaque composant sécurisé.

Au chargement de chaque page, le composant vérifie systématiquement :

- La présence du token JWT dans le `localStorage`.
- Le rôle de l'utilisateur connecté.
- Les permissions nécessaires pour accéder à la fonctionnalité concernée.

Si l'utilisateur ne dispose pas des droits suffisants ou si son token est absent ou expiré, il est automatiquement redirigé vers la page de connexion ou vers le dashboard selon le cas.

L'exemple ci-dessous illustre cette vérification dans le composant de gestion des utilisateurs :



```
14 export class GroupsComponent implements OnInit {
44   ngOnInit(): void {
45     this.loadGroups();
46
47     if (this.isAdmin()) {
48       this.loadUsers();
49     }
50   }
51
52   get role(): string {
53     return localStorage.getItem('role') || '';
54   }
55
56   get currentUserId(): string {
57     return localStorage.getItem('user_id') || '';
58   }
59
60   isAdmin(): boolean {
61     return this.role === 'ADMIN';
62   }
63
64   loadGroups(): void {
65     const token = localStorage.getItem('token');
66
67     if (!token) {
68       this.router.navigate(['/']);
69       return;
70     }
71
72     this.http.get('http://localhost:8000/groups/', {
73       headers: {
74         Authorization: `Bearer ${token}`
75       }
76     }).subscribe({
77       next: (res: any) => {
78         this.groups = res;
79       }
80     });
81   }
82 }
```

Figure 24 : Capture de la protection des accès via le token JWT et les rôles utilisateurs dans Angular

Cette approche présente l'avantage d'être simple, directe et facile à maintenir. Elle garantit que chaque composant est responsable de sa propre sécurité, ce qui réduit les dépendances entre les différentes parties de l'application.

Une amélioration future pourrait consister à centraliser ces vérifications dans des Guards Angular formels afin de réduire la duplication du code et d'améliorer la maintenabilité globale du frontend.

## 4.5 Interfaces et fonctionnalités

Cette section présente les principales interfaces de l'application ainsi que les fonctionnalités mises à disposition des utilisateurs. L'interface utilisateur a été conçue dans une logique de simplicité, de lisibilité et d'efficacité opérationnelle. Une attention particulière a été portée à l'expérience utilisateur, à la clarté des informations affichées et à la cohérence visuelle entre les différentes pages. Le frontend Angular<sup>16</sup> permet une mise à jour dynamique des données sans rechargement complet des pages, améliorant ainsi la fluidité de navigation.

### 4.5.1 Interface d'authentification

L'interface d'authentification constitue le point d'entrée principal de l'application. Elle permet aux utilisateurs de se connecter de manière sécurisée à l'aide de leur adresse e-mail et de leur mot de passe. Après validation des identifiants, un token JWT est généré par le backend et stocké côté frontend afin

<sup>16</sup>Angular : Documentation officielle. Disponible sur : <https://angular.dev>

d'authentifier les requêtes futures. En cas d'identifiants invalides ou de session expirée, un message d'erreur est affiché directement dans la page sans rechargement.

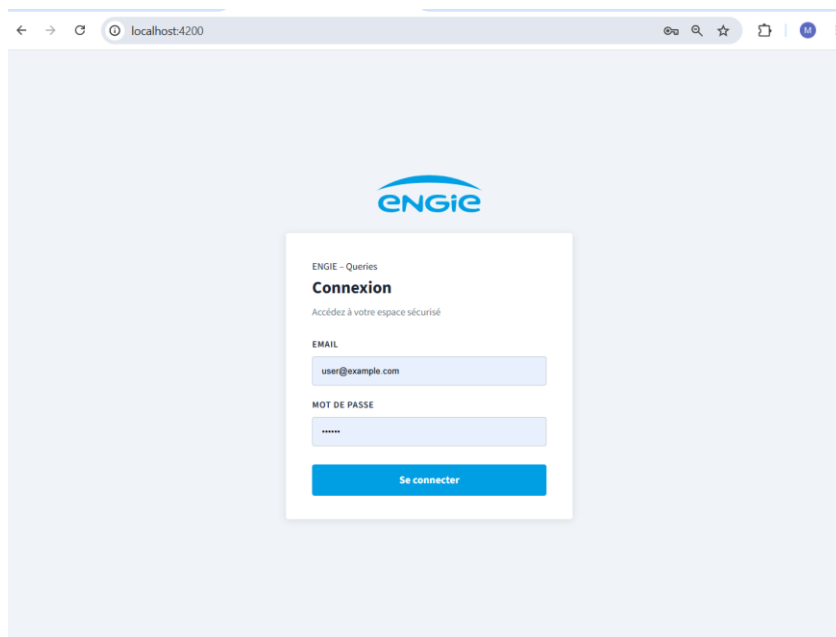


Figure 25 : Capture de l'interface de connexion sécurisée de l'application avec authentification JWT

Les fonctionnalités de cette interface sont les suivantes :

- Authentification sécurisée via JWT.
- Affichage des messages d'erreur en cas d'identifiants invalides.
- Redirection automatique vers le dashboard après connexion réussie.
- Protection des routes et redirection vers la page de connexion en cas de session expirée.

#### 4.5.2 Interface de gestion des utilisateurs

Cette interface est réservée aux administrateurs. Elle permet de gérer l'ensemble des comptes utilisateurs de la plateforme. Les informations affichées pour chaque utilisateur incluent le nom, le prénom, l'adresse e-mail, le rôle et les informations de contact. Un système de recherche permet de filtrer rapidement les utilisateurs par nom, prénom ou adresse e-mail.

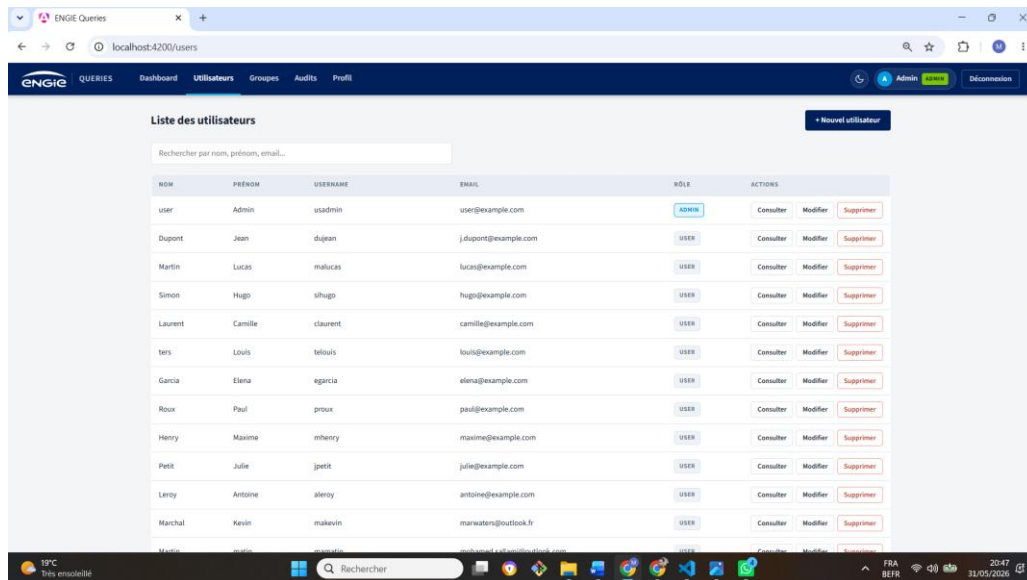


Figure 26 : Capture de l'interface d'administration des utilisateurs de l'application

Les fonctionnalités de cette interface sont les suivantes :

- Création d'un nouvel utilisateur avec validation des champs obligatoires.
- Modification des informations d'un utilisateur existant.
- Consultation du profil détaillé d'un utilisateur et des groupes auxquels il appartient.
- Suppression d'un compte utilisateur avec confirmation via une modale.
- Contrôle des permissions : cette interface est accessible uniquement aux administrateurs.

#### 4.5.3 Interface de gestion des groupes

La gestion des groupes constitue une fonctionnalité centrale de l'application. Cette interface permet de créer des espaces collaboratifs, de consulter les groupes existants et de gérer les membres et les queries associées. Trois filtres de navigation permettent d'afficher tous les groupes, uniquement les groupes dont l'utilisateur est propriétaire ou uniquement les groupes dont il est membre.

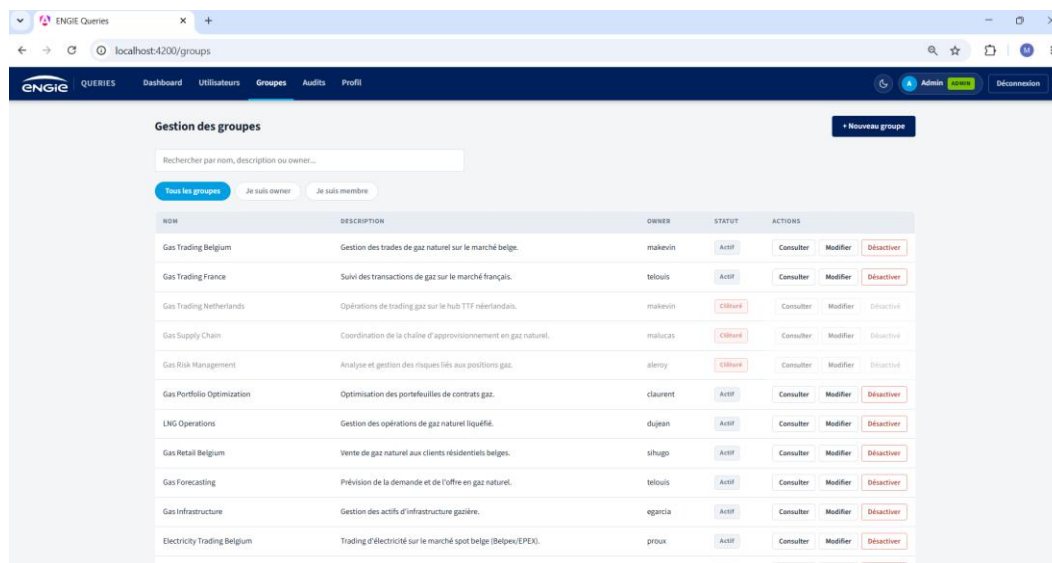


Figure 27 : Capture de l'interface de gestion des groupes de l'application

Les fonctionnalités de cette interface sont les suivantes :

- Création d'un groupe avec nom, description et désignation du propriétaire.
- Modification des informations du groupe par le propriétaire ou l'administrateur.
- Désactivation logique d'un groupe avec envoi automatique d'un email d'alerte au propriétaire.
- Consultation des membres, des queries et de l'historique des audits du groupe.
- Filtrage des groupes par rôle de l'utilisateur connecté.

#### 4.5.4 Interface de gestion des memberships

Cette interface permet de gérer les membres appartenant aux différents groupes. Elle est accessible depuis la page de détail d'un groupe. Les propriétaires et les administrateurs peuvent ajouter ou supprimer des membres et consulter les utilisateurs associés à un groupe. Chaque ajout ou suppression génère automatiquement une entrée d'audit afin de garantir la traçabilité des opérations.

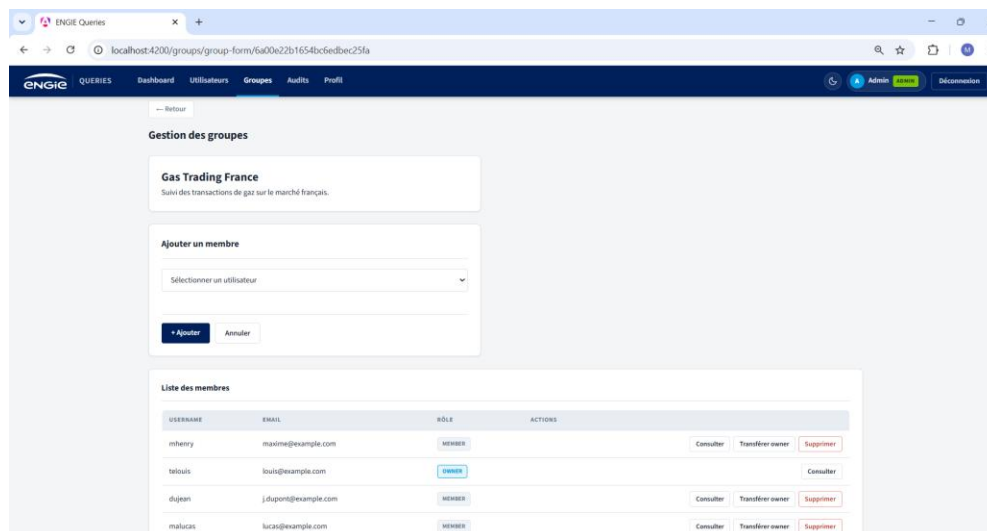


Figure 28 : Capture de l'interface de gestion des membres et des rôles d'un groupe

Les fonctionnalités de cette interface sont les suivantes :

- Ajout d'un membre avec vérification des doublons.
- Suppression d'un membre avec confirmation via une modale.
- Transfert de la propriété du groupe vers un autre membre.
- Génération automatique d'un audit pour chaque opération réalisée.

#### 4.5.5 Interface de gestion des queries

Cette interface permet aux utilisateurs de créer, modifier et gérer des queries d'analyse sur les deals énergétiques. Chaque query est associée à un groupe afin de favoriser le travail collaboratif. Les filtres disponibles permettent de cibler précisément les données souhaitées :

- Le produit énergétique (GAS, ELECTRICITY, OIL, CO2).
- Le portefeuille de trading.
- Les prix minimum et maximum.
- Les dates de début et de fin.
- Le trader et la contrepartie.
- Le statut métier du deal.

Figure 29 : Capture de l'interface de création et de gestion des queries d'analyse des deals énergétiques

Les fonctionnalités de cette interface sont les suivantes :

- Création d'une query avec sélection des filtres et des colonnes à afficher.
- Modification des filtres et des paramètres d'une query existante.
- Suppression d'une query avec confirmation.
- Liaison obligatoire de chaque query à un groupe.

#### 4.5.6 Interface d'exécution des queries et résultats

Cette interface permet d'exécuter une query et d'afficher les résultats sous forme de tableau dynamique. Les données sont récupérées en temps réel depuis MongoDB via les APIs REST du backend FastAPI en appliquant les filtres définis dans la query. Un résumé statistique est affiché en haut des résultats, indiquant le nombre de deals trouvés, la quantité totale, le montant total et le prix moyen pondéré.

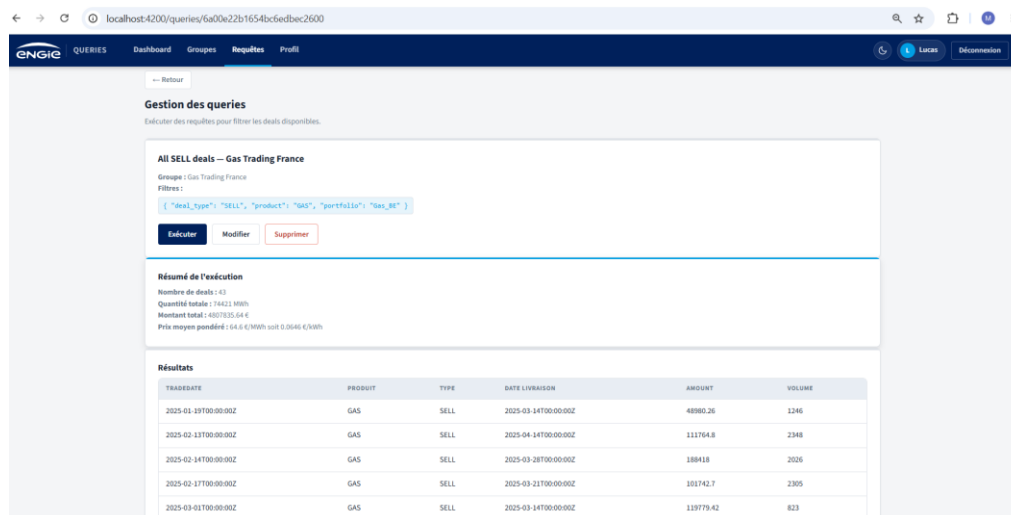


Figure 30 : Capture de l'interface d'exécution des queries et d'affichage des résultats des deals énergétiques

Les fonctionnalités de cette interface sont les suivantes :

- Exécution dynamique des filtres sur la collection deals.
- Affichage des résultats sous forme de tableau avec les colonnes sélectionnées.
- Résumé statistique des résultats (nombre de deals, volume total, montant total, prix moyen pondéré).
- Modification des filtres directement depuis l'interface d'exécution.

#### 4.5.7 Interface d'audit et de traçabilité

L'interface d'audit permet de consulter l'ensemble des opérations importantes réalisées dans l'application. Elle est accessible depuis la page de détail d'un groupe via le bouton Historique. Les logs d'audit sont consultables par les administrateurs et par les membres du groupe concerné. Chaque entrée affiche l'utilisateur responsable, le type d'action, l'horodatage, l'entité concernée ainsi que les anciennes et nouvelles valeurs lorsque cela est pertinent.

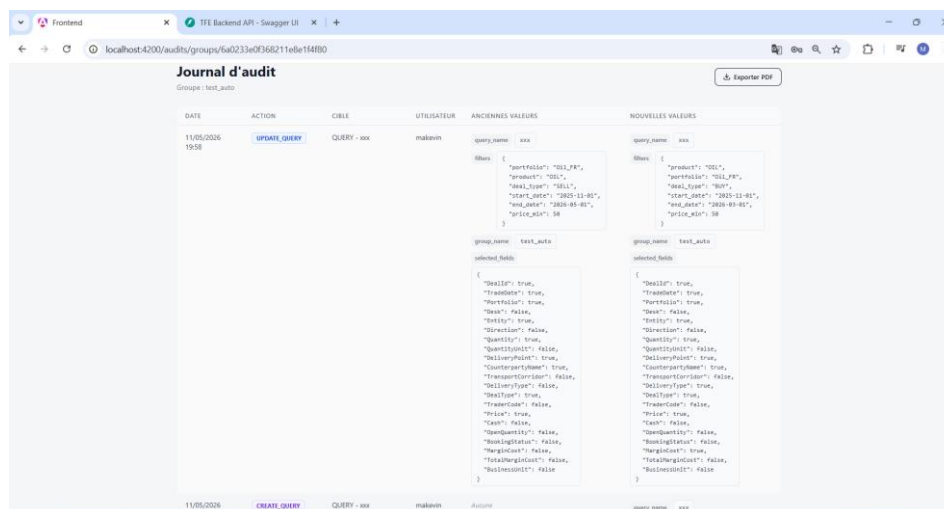


Figure 31 : Capture de l'interface d'audit et de traçabilité des actions réalisées dans l'application

Les fonctionnalités de cette interface sont les suivantes :

- Consultation de l'historique complet des actions par groupe.
- Affichage des anciennes et nouvelles valeurs pour chaque modification.
- Accès restreint aux membres du groupe et aux administrateurs.
- Logs en lecture seule (aucune modification n'est possible).

#### 4.5.8 Documentation Swagger

Le backend FastAPI génère automatiquement une documentation interactive grâce à Swagger, accessible à l'adresse /docs. Cette interface permet de :

- Consulter l'ensemble des endpoints REST disponibles.
- Tester les APIs directement depuis le navigateur.
- Visualiser les paramètres attendus et les réponses JSON générées par le backend.
- Faciliter le débogage et la validation des fonctionnalités.

Swagger a été largement utilisé durant le développement afin de tester les fonctionnalités, valider les endpoints et faciliter l'intégration entre le frontend et le backend. Grâce à cette documentation automatique, il n'est pas nécessaire de maintenir manuellement une documentation séparée car elle est toujours à jour et reflète exactement l'état réel des APIs.

#### 4.6 Tests fonctionnels

Afin de garantir le bon fonctionnement de l'application et la conformité aux règles métier définies dans le cahier des charges, une campagne de tests fonctionnels manuels a été réalisée tout au long du développement. Chaque fonctionnalité a été testée directement via l'interface, en se plaçant dans la peau des différents profils d'utilisateurs (administrateur, owner, member), afin de vérifier à la fois le comportement attendu et le respect des permissions.

Les tests ont également porté sur les cas d'erreur (identifiants invalides, accès non autorisé, champs obligatoires manquants) afin de s'assurer que l'application réagit correctement et affiche des messages



clairs à l'utilisateur. Lorsqu'une anomalie était détectée, elle était corrigée puis le scénario était rejoué pour validation.

Tableau 9 : Les principaux scénarios de tests fonctionnels réalisés

| N° | Scénario testé                    | Action réalisée                                                                   | Résultat attendu                                | Status |
|----|-----------------------------------|-----------------------------------------------------------------------------------|-------------------------------------------------|--------|
| 1  | Connexion valide                  | Saisir email + mot de passe corrects                                              | Redirection vers le dashboard                   | ✓      |
| 2  | Connexion invalide                | Saisir un mauvais mot de passe                                                    | Message d'erreur, pas d'accès                   | ✓      |
| 3  | Accès sans connexion              | Ouvrir une page protégée sans être connecté                                       | Redirection vers la page de connexion           | ✓      |
| 4  | Création d'un utilisateur (admin) | Remplir le formulaire utilisateur                                                 | Utilisateur créé et visible dans la liste       | ✓      |
| 5  | Restriction d'accès               | Se connecter en tant que MEMBER et tenter d'accéder à la gestion des utilisateurs | Accès refusé                                    | ✓      |
| 6  | Création d'un groupe              | Créer un groupe via le formulaire                                                 | Groupe créé, créateur devient OWNER             | ✓      |
| 7  | Ajout d'un membre                 | Ajouter un utilisateur à un groupe                                                | Membre ajouté, audit généré                     | ✓      |
| 8  | Ajout d'un doublon                | Ajouter deux fois le même membre                                                  | Ajout refusé (message d'erreur)                 | ✓      |
| 9  | Création et exécution d'une query | Créer une query avec filtres puis l'exécuter                                      | Résultats filtrés + résumé statistique affichés | ✓      |
| 10 | Restriction admin sur les queries | Se connecter en admin et tenter de créer une query                                | Action refusée                                  | ✓      |
| 11 | Désactivation d'un groupe         | Désactiver un groupe en tant qu'owner                                             | Groupe désactivé + email d'alerte envoyé        | ✓      |
| 12 | Consultation des audits           | Ouvrir l'historique d'un groupe                                                   | Liste des actions affichée en lecture seule     | ✓      |
| 13 | Reporting automatique             | Vérifier la génération et l'envoi du PDF d'audit                                  | PDF généré et reçu par email                    | ✓      |

Ces tests fonctionnels m'ont permis de valider l'ensemble des user stories du projet et de garantir que les principales fonctionnalités, ainsi que les contrôles de permissions, fonctionnent comme prévu. Une amélioration future consisterait à automatiser ces tests afin de détecter plus rapidement d'éventuelles régressions.

## ■ 4.7 Difficultés rencontrées

Le développement de cette application a représenté un véritable défi technique, combinant plusieurs technologies modernes dans un contexte professionnel exigeant. Les principales difficultés rencontrées sont présentées ci-dessous.

### **Gestion des ObjectId MongoDB**

L'une des difficultés les plus récurrentes du projet concernait la gestion des identifiants MongoDB. Les ObjectId sont des objets spécifiques à MongoDB qui ne peuvent pas être sérialisés directement en JSON. Il a fallu gérer les conversions entre les ObjectId MongoDB, les chaînes de caractères côté backend et les données reçues par le frontend Angular. Des erreurs de permissions ou de recherches apparaissaient fréquemment lorsque ces conversions n'étaient pas correctement gérées. Cette difficulté a nécessité plusieurs ajustements techniques afin de garantir la cohérence des données entre les trois couches de l'application.

### **Communication frontend/backend et gestion des erreurs**

L'intégration entre Angular et FastAPI a demandé une attention particulière. Plusieurs difficultés ont été rencontrées concernant la configuration CORS, les formats JSON, la gestion des erreurs HTTP et les validations croisées backend/frontend. L'utilisation de Swagger et Postman a permis de faciliter les tests et la validation des endpoints REST, mais certaines erreurs subtiles liées aux types de données ont nécessité plusieurs corrections.

### **Mise en place du scheduler APScheduler**

La configuration du scheduler APScheduler pour la génération automatique des rapports PDF a représenté une difficulté technique importante. Il a fallu s'assurer que le scheduler se lance correctement au démarrage de l'application via le mécanisme lifespan de FastAPI, qu'il s'arrête proprement à l'extinction et qu'il gère correctement les fuseaux horaires pour le déclenchement mensuel. La configuration de l'envoi des emails via SMTP et la génération des PDF via ReportLab<sup>17</sup> ont également nécessité plusieurs ajustements.

---

<sup>17</sup>ReportLab : Documentation officielle. Disponible sur : <https://www.reportlab.com/docs/reportlab-userguide.pdf>

## 5. CONCLUSION

---

Ce travail de fin d'études a représenté pour moi une expérience particulièrement enrichissante, tant sur le plan technique que professionnel. Il m'a permis de reconcevoir et de développer une application web sécurisée, en repartant de la base existante chez ENGIE pour en faire une solution complète, collaborative et adaptée aux exigences d'un environnement professionnel réel.

Le projet m'a donné l'opportunité de mettre en pratique l'ensemble des compétences acquises durant ma formation en informatique de gestion, en les confrontant aux contraintes concrètes d'un projet d'entreprise. La combinaison de technologies modernes, notamment Angular, FastAPI, MongoDB, JWT et APScheduler, m'a permis de développer une vision globale des architectures applicatives actuelles et des bonnes pratiques associées.

Les aspects qui m'ont le plus apporté sur le plan technique sont la conception du système de gestion des rôles RBAC, la mise en place du système d'audit et de traçabilité, ainsi que le développement du mécanisme de reporting automatique. Ces fonctionnalités m'ont permis de mieux comprendre l'importance de la sécurité, du contrôle des accès et de la traçabilité dans les applications manipulant des données sensibles.

Ce projet m'a également confrontée à des difficultés techniques réelles telles que la gestion des ObjectId en MongoDB, la configuration du scheduler APScheduler, la synchronisation des permissions entre Angular et FastAPI. Ces défis ont renforcé ma capacité d'analyse, mon autonomie et ma logique de développement.

Au-delà des aspects techniques, cette expérience m'a permis de découvrir le fonctionnement d'un environnement professionnel exigeant, de travailler en méthodologie Agile et d'utiliser des outils professionnels tels que Git, Swagger, Postman et Microsoft Teams.

Plusieurs améliorations pourraient être envisagées pour faire évoluer l'application dans le futur :

- La mise en place de Guards Angular formels afin de centraliser la protection des routes.
- L'harmonisation des types de références dans MongoDB, en uniformisant le stockage des identifiants entre le type String et le type ObjectId.
- L'ajout de tableaux de bord analytiques avec graphiques et statistiques.
- L'intégration d'exports Excel avancés pour les résultats des queries.
- La mise en place d'un système de notifications en temps réel.
- Le développement d'une architecture microservices pour améliorer la scalabilité.

Ce travail de fin d'études constitue une synthèse concrète de mon parcours académique et une première expérience professionnelle significative. Il m'a confirmé mon intérêt pour le développement backend et les architectures distribuées, et m'a donné envie de continuer à approfondir ces domaines dans ma carrière professionnelle.

## 6. BIBLIOGRAPHIE

---

- Angular : Documentation officielle. Google. Disponible sur : <https://angular.dev>
- Angular : Guide des Standalone Components. Disponible sur : <https://angular.dev/guide/components>
- APScheduler : Documentation officielle. Disponible sur : <https://apscheduler.readthedocs.io>
- bcrypt : Hachage sécurisé des mots de passe. Disponible sur : <https://pypi.org/project/bcrypt>
- Draw.io : Outil de modélisation UML. Disponible sur : <https://app.diagrams.net>
- FastAPI : Documentation officielle. Tiangolo, S. Disponible sur : <https://fastapi.tiangolo.com>
- FastAPI : Sécurisation des routes avec OAuth2 et JWT. Disponible sur : <https://fastapi.tiangolo.com/tutorial/security/oauth2-jwt>
- Git : Documentation officielle. Disponible sur : <https://git-scm.com/doc>
- JWT : Introduction aux JSON Web Tokens. Disponible sur : <https://jwt.io/introduction>
- MongoDB Compass : Interface graphique MongoDB. Disponible sur : <https://www.mongodb.com/products/compass>
- MongoDB : Documentation officielle. MongoDB Inc. Disponible sur : <https://www.mongodb.com/docs>
- MongoDB : Introduction aux bases de données NoSQL. Disponible sur : <https://www.mongodb.com/nosql-explained>
- Passlib : Documentation officielle. Disponible sur : <https://passlib.readthedocs.io>
- Postman : Outil de test des APIs. Disponible sur : <https://www.postman.com>
- Pydantic : Documentation officielle. Disponible sur : <https://docs.pydantic.dev>
- PyMongo : Documentation officielle. MongoDB Inc. Disponible sur : <https://pymongo.readthedocs.io>
- Python 3.11 : Documentation officielle. Disponible sur : <https://docs.python.org/3.11>
- Python-jose : Documentation officielle. Disponible sur : <https://python-jose.readthedocs.io>
- RBAC : Role-Based Access Control. Disponible sur : <https://auth0.com/docs/manage-users/access-control/rbac>
- ReportLab : Documentation officielle. Disponible sur : <https://www.reportlab.com/docs/reportlab-userguide.pdf>
- Swagger UI : Documentation interactive des APIs REST. Disponible sur : <https://swagger.io/tools/swagger-ui>

## TABLE DES FIGURES

|                                                                                                                                                                |    |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| <b>Figure 1</b> : Capture du planning des sprints et du backlog utilisés dans le cadre de la méthodologie Agile Scrum .....                                    | 9  |
| <b>Figure 2</b> : Diagramme de cas d'utilisation illustrant les fonctionnalités accessibles aux utilisateurs, owners et administrateurs de l'application ..... | 13 |
| <b>Figure 3</b> : Rapport PDF généré automatiquement par l'application .....                                                                                   | 15 |
| <b>Figure 4</b> : Diagramme UML de la structure de l'application représentant les principales entités du système .....                                         | 16 |
| <b>Figure 5</b> : Organisation des collections MongoDB de l'application représentant les différentes entités métier .....                                      | 21 |
| <b>Figure 6</b> : Document d'un utilisateur stocké dans la collection users de MongoDB .....                                                                   | 21 |
| <b>Figure 7</b> : Document d'un groupe enregistré dans la collection groups de MongoDB .....                                                                   | 22 |
| <b>Figure 8</b> : Documents enregistrés dans la collection memberships de MongoDB .....                                                                        | 22 |
| <b>Figure 9</b> : Document enregistré dans la collection queries de MongoDB .....                                                                              | 23 |
| <b>Figure 10</b> : Document enregistré dans la collection deals de MongoDB.....                                                                                | 23 |
| <b>Figure 11</b> : Document enregistré dans la collection audits de MongoDB .....                                                                              | 24 |
| <b>Figure 12</b> : Diagramme de séquence illustrant le processus d'authentification sécurisé de l'application                                                  | 25 |
| <b>Figure 13</b> : Diagramme de séquence illustrant le processus de génération automatique des rapport d'audit PDF .....                                       | 26 |
| <b>Figure 14</b> : Diagrammes d'activités illustrant les principaux processus métier de l'application.....                                                     | 27 |
| <b>Figure 15</b> : Diagramme BPMN illustrant le fonctionnement global de l'application.....                                                                    | 28 |
| <b>Figure 16</b> : Schéma d'architecture de l'application .....                                                                                                | 29 |
| <b>Figure 17</b> : Capture du fichier app.routes.ts illustrant la configuration des routes Angular de l'application .....                                      | 30 |
| <b>Figure 18</b> : Capture de l'organisation du backend FastAPI .....                                                                                          | 32 |
| <b>Figure 19</b> : Capture de la documentation Swagger des endpoints sécurisés du backend FastAPI .....                                                        | 34 |
| <b>Figure 20</b> : Organisation du projet sur GitHub avec gestion des branches et historique des commits. ....                                                 | 35 |
| <b>Figure 21</b> : Capture de la configuration SMTP .....                                                                                                      | 37 |
| <b>Figure 22</b> : Capture des filtres dynamiques MongoDB pour la recherche des deals énergétiques .....                                                       | 38 |
| <b>Figure 23</b> : Capture de l'implémentation de la génération automatique des audits dans le backend FastAPI.....                                            | 39 |
| <b>Figure 24</b> : Capture de la protection des accès via le token JWT et les rôles utilisateurs dans Angular ....                                             | 42 |
| <b>Figure 25</b> : Capture de l'interface de connexion sécurisée de l'application avec authentification JWT ....                                               | 43 |
| <b>Figure 26</b> : Capture de l'interface d'administration des utilisateurs de l'application .....                                                             | 44 |
| <b>Figure 27</b> : Capture de l'interface de gestion des groupes de l'application .....                                                                        | 45 |

|                                                                                                                             |           |
|-----------------------------------------------------------------------------------------------------------------------------|-----------|
| <b>Figure 28</b> : Capture de l'interface de gestion des membres et des rôles d'un groupe .....                             | <b>45</b> |
| <b>Figure 29</b> : Capture de l'interface de création et de gestion des queries d'analyse des deals énergétiques .....      | <b>46</b> |
| <b>Figure 30</b> : Capture de l'interface d'exécution des queries et d'affichage des résultats des deals énergétiques ..... | <b>47</b> |
| <b>Figure 31</b> : Capture de l'interface d'audit et de traçabilité des actions réalisées dans l'application.....           | <b>48</b> |

## LISTE DES TABLEAUX

---

|                                                                                 |           |
|---------------------------------------------------------------------------------|-----------|
| <b>Tableau 1</b> : Tableau des permissions des utilisateurs .....               | <b>12</b> |
| <b>Tableau 2</b> : Collection users.....                                        | <b>18</b> |
| <b>Tableau 3</b> : Collection groups .....                                      | <b>19</b> |
| <b>Tableau 4</b> : Collection memberships .....                                 | <b>19</b> |
| <b>Tableau 5</b> : Collection queries .....                                     | <b>19</b> |
| <b>Tableau 6</b> : Collection deals.....                                        | <b>19</b> |
| <b>Tableau 7</b> : Collection audits .....                                      | <b>20</b> |
| <b>Tableau 8</b> : Outils utilisés durant le développement du projet.....       | <b>34</b> |
| <b>Tableau 9</b> : Les principaux scénarios de tests fonctionnels réalisés..... | <b>49</b> |